

Datenbanken I

Andreas Nicola

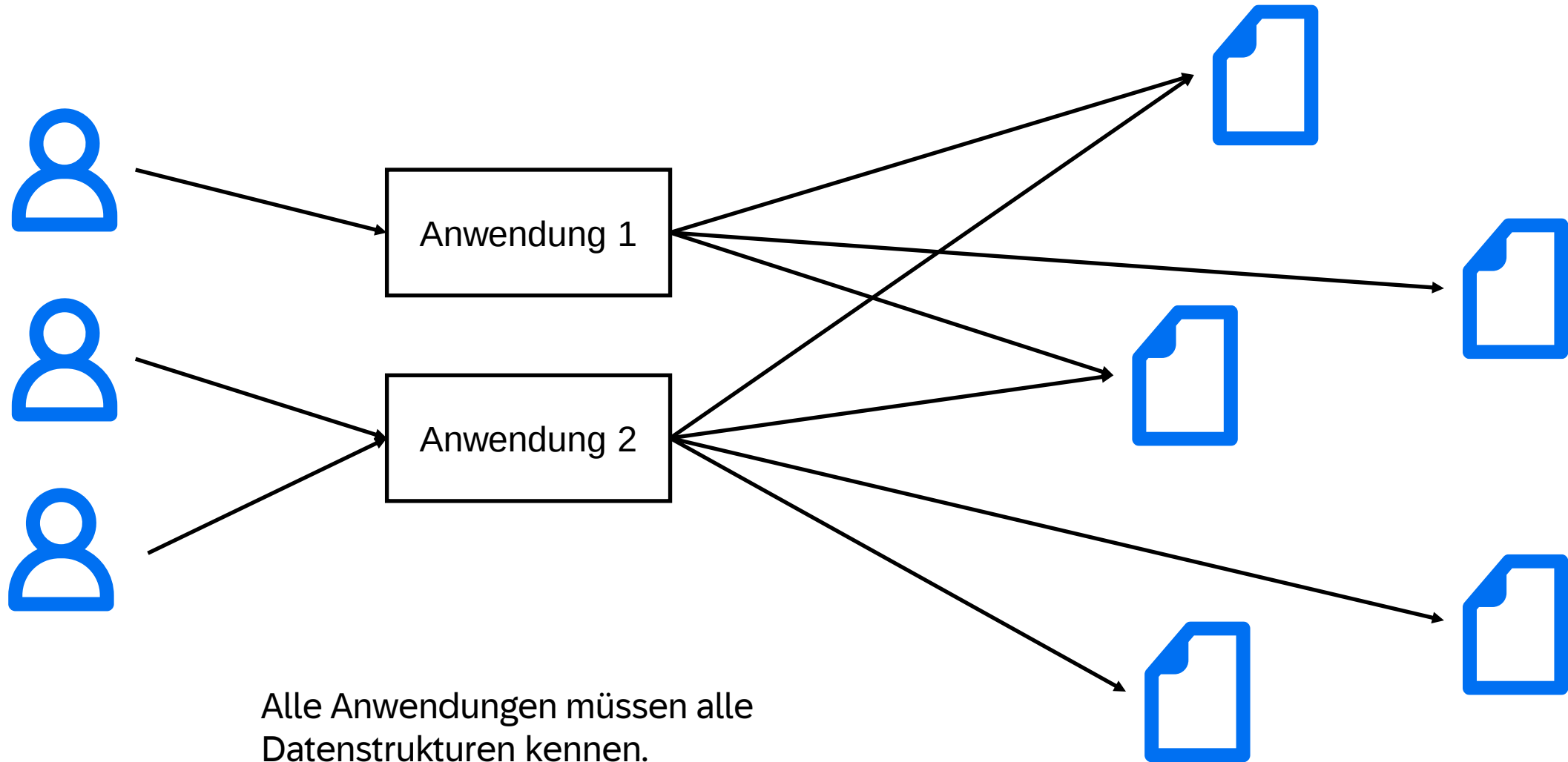
Mail: andreas.nicola@sap.com

Agenda

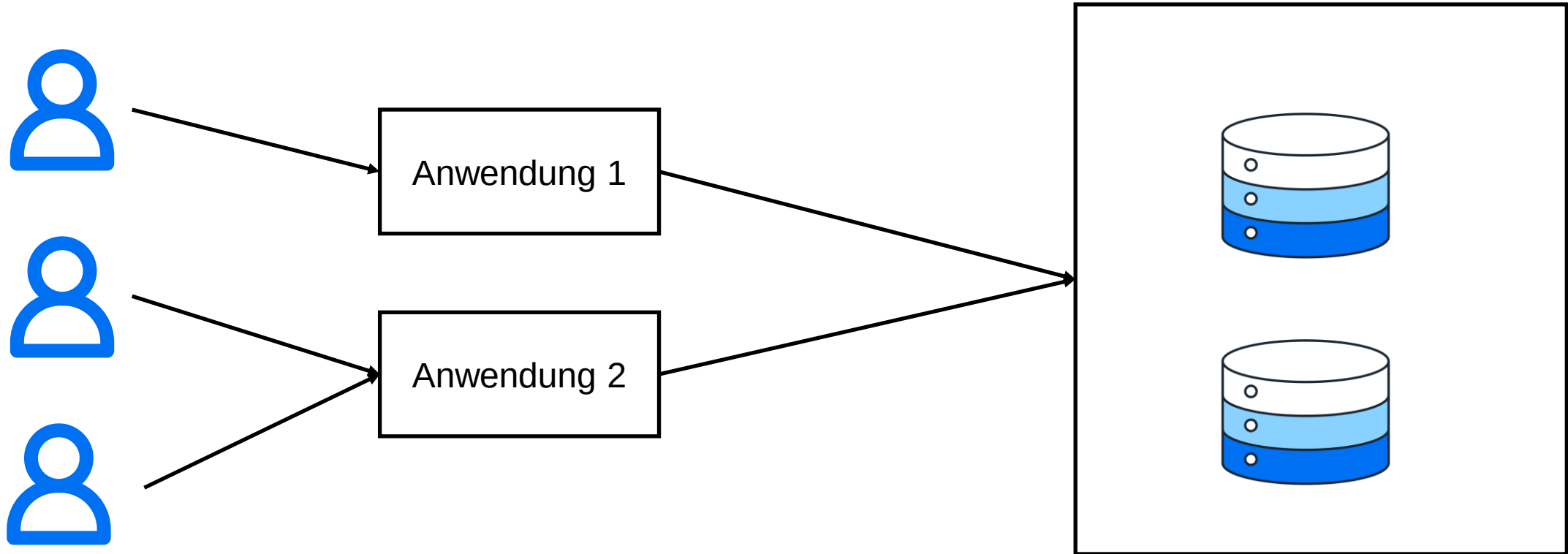
- Datenbankbegriffe
- Datenbankmodelle
- Funktionen
- Transaktionen
- ACID
- Relationale Algebra
- Entity-Relationship-Modell
- Relationenmodell
- Normalisierung
- SQL – Structured Query Language

Datenbanksystem

Datenorganisation



Datenbank



Eine Schnittstelle und Datenstruktur
für alle Anwendungen.

Begriffe

Datenbank

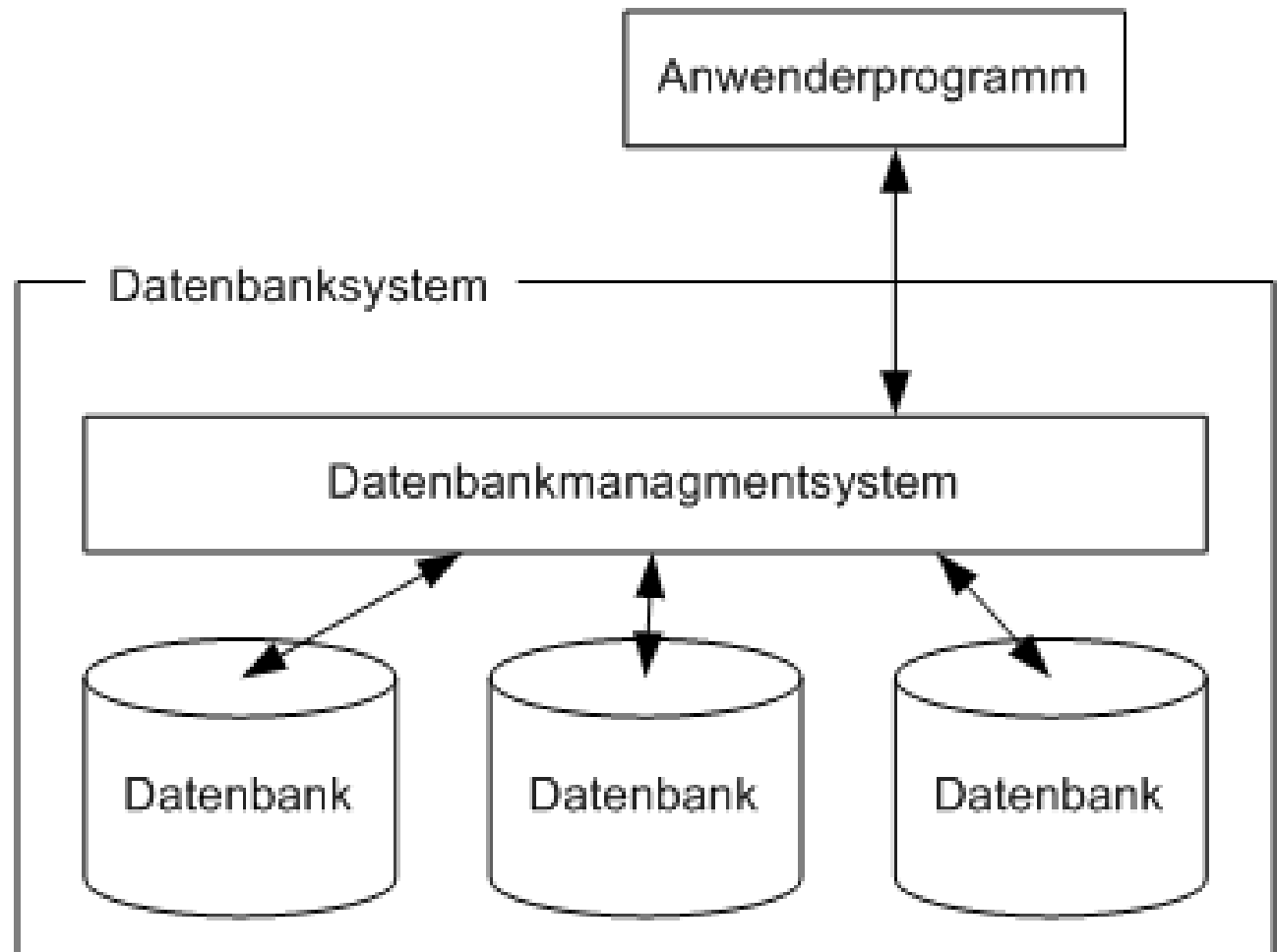
Strukturierte Sammlung von Daten

Datenbankmanagementsystem

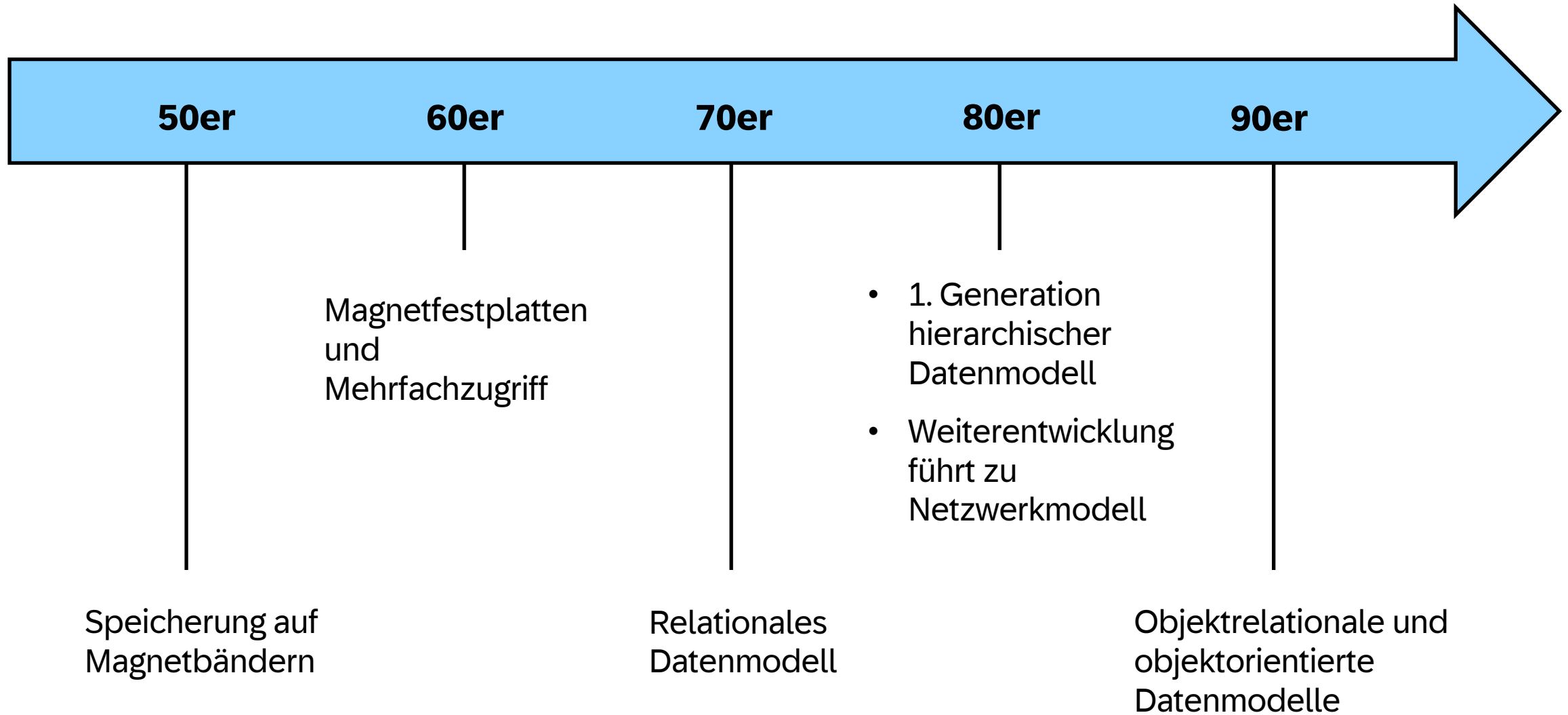
Software zur Verwaltung der Datenbank

Datenbanksystem

Kombination aus Datenbank und Datenbankmanagementsystem

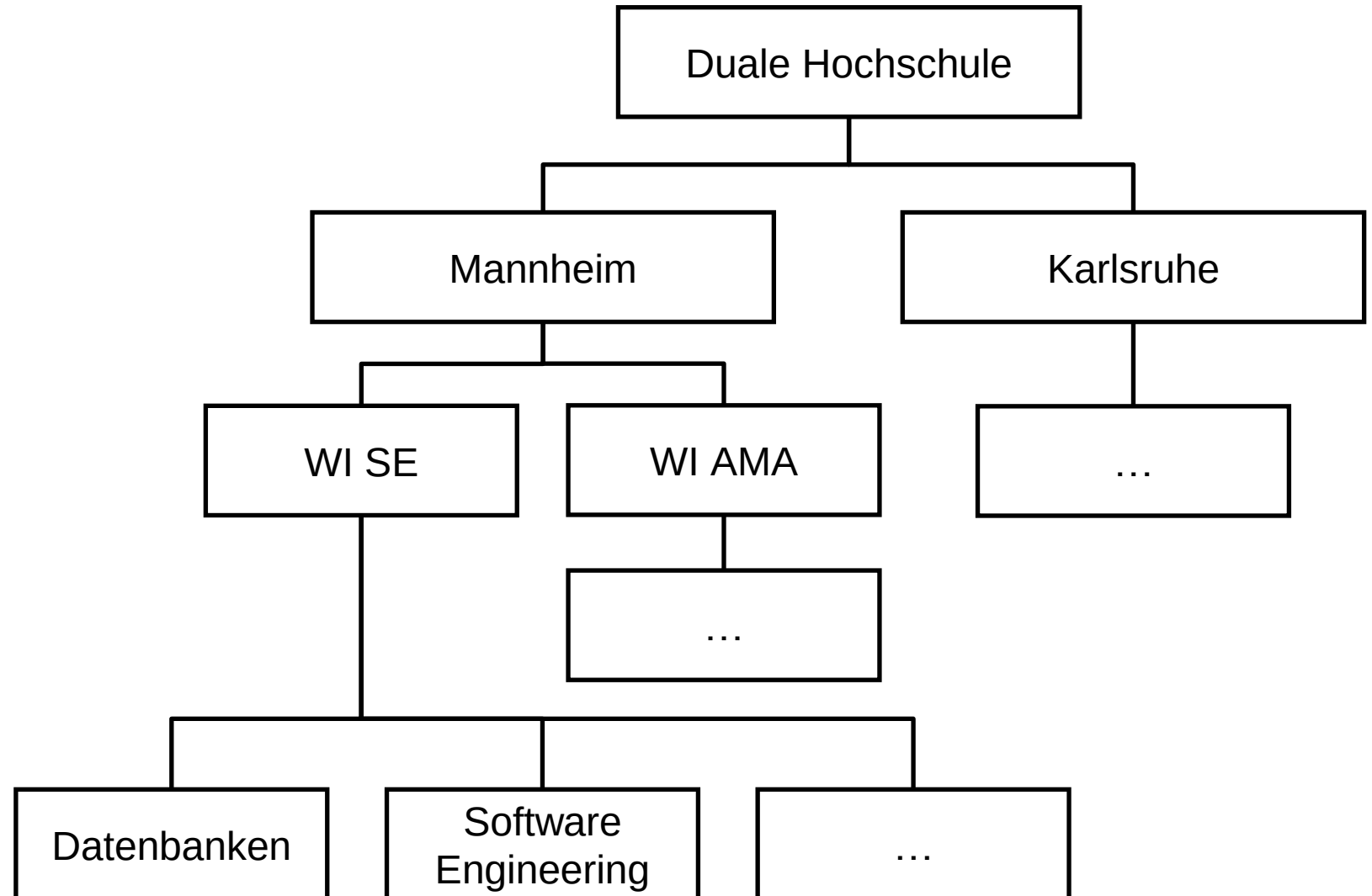


Historie



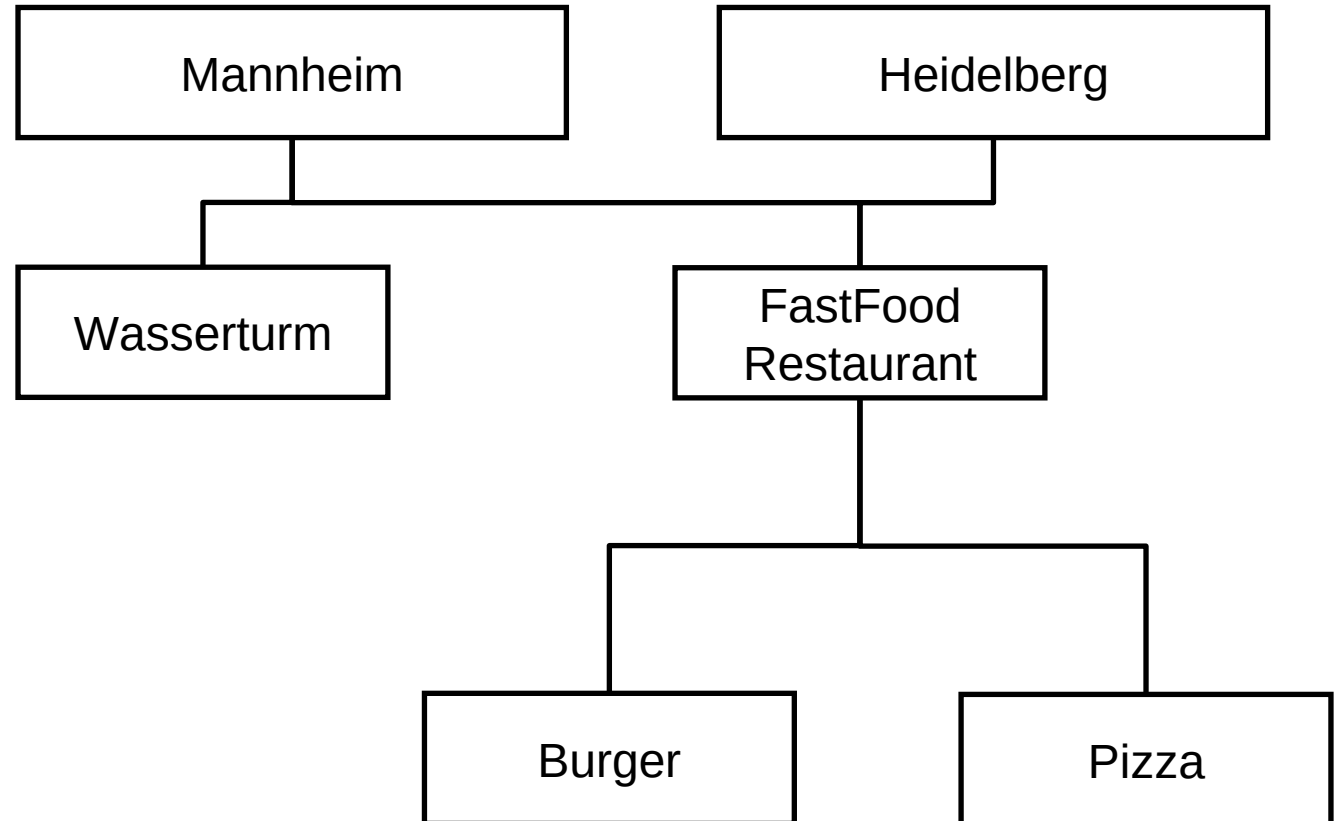
Hierarchisches Datenbankmodell

- Baumstruktur
- Eltern-Kind (Parent-Child) Beziehungen
- Ein Child kann nur einen Parent besitzen, ein Parent beliebig viele Children.



Netzwerk Datenbankmodell

- Erweiterung Hierarchisches Datenbankmodell
- Ein Child kann mehrere Parents besitzen, ein Parent beliebig viele Children.



Relationales Datenbankmodell

- Speicherung in Tabellen (Relationen)
- Beziehungen zwischen Tabellen
- Verbindung über Attribute

Matrikelnummer	Kurs
123	Datenbanken
456	Datenbanken
789	Software Engineering

Matrikelnummer	Vorname	Nachname
123	Max	Meier
456	Maria	Müller
789	Beate	Bauer

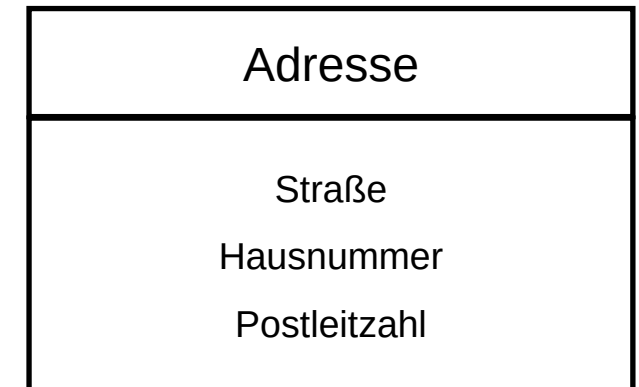
Objektorientiertes Datenbankmodell

- Speicherung als Objekte
- Unterstützung von Klassen, Methoden, Vererbung

Objektrelationales Datenbankmodell

- Kombination aus relationalem und objektorientierten Modell

Vorname	Nachname	Adresse
Max	Meier	<Adressobjekt>
Maria	Müller	<Adressobjekt>
Beate	Bauer	<Adressobjekt>



Dokumentenorientiertes Datenbankmodell

- Speicherung als Dokumente (JSON, XML, CSV, etc.)
- Kein festes Schema, flexibel

```
{  
  "Kunde": {  
    "Name": "Lisa",  
    "Adresse": "Berlin",  
    "Bestellungen": [  
      {"Produkt": "Laptop", "Preis": 1200},  
      {"Produkt": "Handy", "Preis": 800}  
    ]  
  }  
}
```

JSON Dokument

Funktionen DBMS

- Redundanzfreiheit
 - Datenintegrität
 - Datensicherheit
 - Datenschutz
 - Datenunabhängigkeit
 - Mehrbenutzerbetrieb
 - Datenspeicherung / Datenmanipulation
 - Technische Kennzahlen
 - Skalierbarkeit
 - Schnelle Zugriffszeiten
- Hierarchisches Modell
 - IBM Information Management System
 - Netzwerk Modell
 - Integrated Data Management System
 - Relationales Modell
 - PostgreSQL, MySQL, Oracle Database, HANA
 - Objektrelationales Modell
 - PostgreSQL, Oracle ORDBMS
 - Objektorientiertes Modell
 - ObjectDB
 - Dokumentenorientiertes Modell
 - MongoDB, CouchDB

Transaktionen

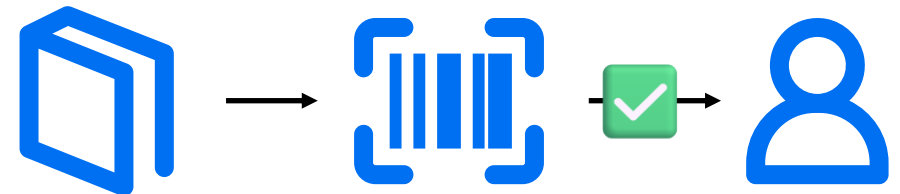
- Logische Einheit von Abläufen (mehrere Schritte, ggf. Änderungen)
- Konsistente Operation auf einer Datenbank
- Abkapselung verschiedener Benutzer

Erfolg: COMMIT
Fehler: ROLLBACK

ROLLBACK → Änderungen werden rückgängig gemacht



Buch ausgeliehen?



COMMIT → Änderungen werden durchgeführt und gespeichert

ACID

A → Atomicity (Atomarität)

Vollständige oder keine Ausführung, Fehler führt dazu, dass alle bis dahin ausgeführten Schritte rückgängig gemacht werden.



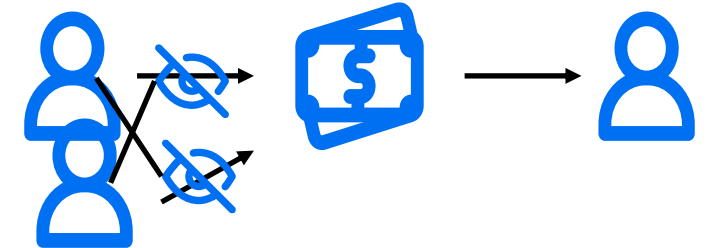
C → Consistency (Konsistenz)

Einhaltung Integrität und Referenzielle Integrität, konsistenter Zustand davor und danach.



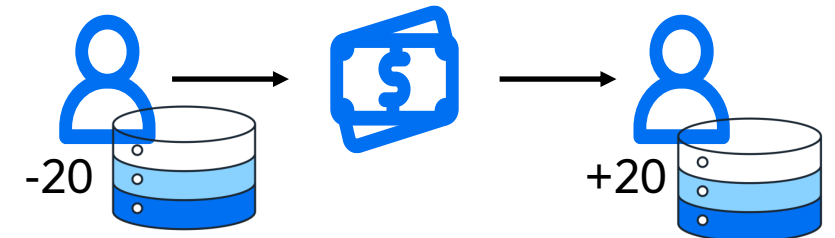
I → Isolation (Isolation)

Keine Sichtbarkeit für andere Transaktionen, Einschränkungen z.B. durch Read Committed.



D → Durability (Dauerhaftigkeit)

Nach erfolgreicher Transaktion sind die Daten dauerhaft in permanentem Speicher gespeichert.



ANSI/SPARC Architektur – Drei-Ebenen-Architektur

- Entwicklung 1975 von Standards Planning and Requirements Committee (SPARC) und American National Standards Institute (ANSI)
- Datenunabhängigkeit durch Trennung physischer Speicherung und Verwaltung

Physische Datenunabhängigkeit

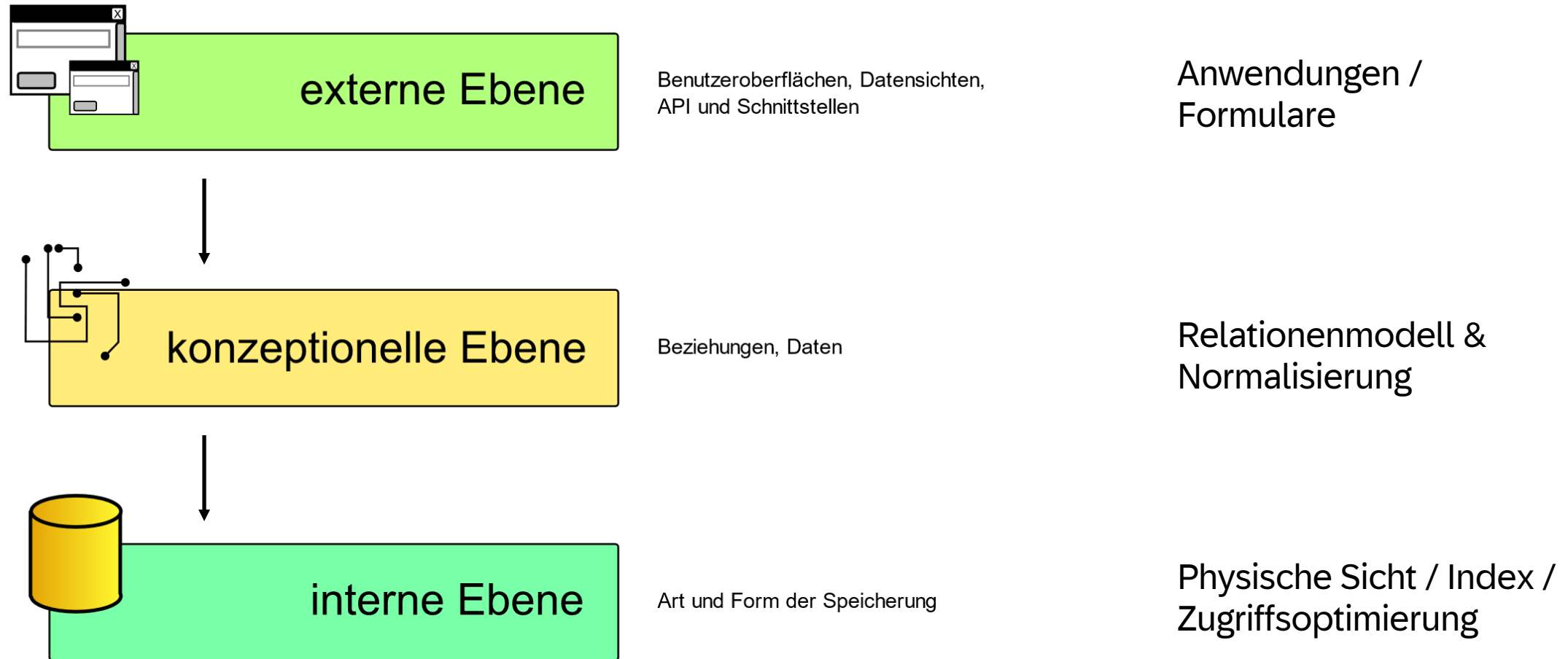
Strukturänderungen, Hardware-Änderungen, Änderung Datenbanksystem

Logische Datenunabhängigkeit

Trennung zwischen Struktur und Anwendung (Sicht)

Sicht → Ein Ausschnitt einer Datenbank, der die für eine Anwendung bzw. ein Problem relevanten Daten enthält.

ANSI/SPARC Architektur – Drei-Ebenen-Architektur



Relationale Algebra

Relationale Algebra

	Operator	Beispiel
Vereinigung	$\cup$	$R1 \cup R2$
Schnitt	$\cap$	$R1 \cap R2$
Differenz	$\setminus$	$R1 \setminus R2$
Kreuzprodukt	$\times$	$R1 \times R2$
Restriktion	σ	$\sigma_{\text{Bedingung}}(R)$
Projektion	π	$\pi_{\text{Auswahl}}(R)$
Verbund	$\bowtie$	$R1 \bowtie R2$
Division	$\div$	$R1 \div R2$
Umbenennung	ρ	$\rho_{R_{\text{neu}}}(R)$

$R1 / R2 \rightarrow$ Relationen (Tabellen)

Edwin Schicker: Datenbanken und SQL (2)

Relationale Algebra – Restriktion / Selektion

R

K_ID	Name	Vorname	Geb. Datum
1	Meier	Moritz	01.01.2000
2	Müller	Max	01.01.1990
3	Mustermann	Mark	01.01.1950

Reduzierung der Zeilen durch
Einschränkung (hier: nicht älter als X Jahre)

R

K_ID	Name	Vorname	Geb. Datum
1	Meier	Moritz	01.01.2000
2	Müller	Max	01.01.1990

Relationale Algebra – Projektion

R

K_ID	Name	Vorname	Geb. Datum
1	Meier	Moritz	01.01.2000
2	Müller	Max	01.01.1990
3	Mustermann	Mark	01.01.1950

Einschränkung der Spalten

R

K_ID	Name	Vorname
1	Meier	Moritz
2	Müller	Max
3	Mustermann	Mark

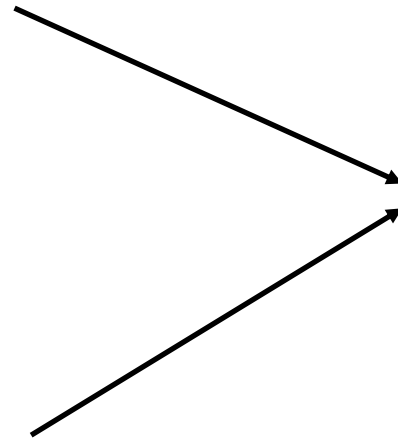
Relationale Algebra – Kreuzprodukt

R

AA	BB
A1	B1
A2	B2

S

CC	DD
C1	D1
C2	D2



AA	BB	CC	DD
A1	B1	C1	D1
A1	B1	C2	D2
A2	B2	C1	D1
A2	B2	C2	D2

Kombination jeder Zeile
aus **R** mit jeder Zeile aus **S**

Relationale Algebra – Vereinigung

Voraussetzung, **R** und **S** haben die gleiche Struktur → Anzahl Attribute und deren Wertebereich

R

Vorname	Nachname
Max	Meier
Marie	Müller

S

Vorname	Nachname
Marie	Müller
Beate	Bauer

Vorname	Nachname
Max	Meier
Marie	Müller
Beate	Bauer

Vereinigung aller Attribute,
doppelte Einträge werden
entfernt

Relationale Algebra – Schnitt

R

Vorname	Nachname
Max	Meier
Marie	Müller

S

Vorname	Nachname
Marie	Müller
Beate	Bauer

Voraussetzung → **Vereinigung**

Vorname	Nachname
Marie	Müller

Schnittmenge aus **R** und **S**

Relationale Algebra – Differenz

Voraussetzung → **Vereinigung**

R

Vorname	Nachname
Max	Meier
Marie	Müller

S

Vorname	Nachname
Marie	Müller
Beate	Bauer

Vorname	Nachname
Max	Meier

Entfernung aller Datensätze aus **R**,
welche in **S** auch vorhanden sind

Relationale Algebra – Verbund / Join

R

Vorname	Nachname	Kurs
Max	Meier	WI SE
Marie	Müller	WI SE

S

Kurs	Beschreibung
WI SE	Wirtschaftsinformatik – Software Engineering

Verknüpfung von **R** und **S**
basierend auf gleichwertigen
Attributen (hier: „Kurs“)

Anreicherung von **R** durch
Informationen aus **S**,
Verknüpfungsattribute werden
nur einmal aufgeführt

Vorname	Nachname	Kurs	Beschreibung
Max	Meier	WI SE	Wirtschaftsinformatik – Software Engineering
Marie	Müller	WI SE	Wirtschaftsinformatik – Software Engineering

Relationale Algebra – Division

Vorname	Nachname	Artikel	Preis
Max	Meier	Buch	15
Marie	Müller	Buch	15
Max	Meier	Film	15
Marie	Müller	Film	10
Bella	Becker	Film	15

R

Datensätze, welche in jeder
Kombination aus **S** in **R** vorkommen

Artikel	Preis
Buch	15
Film	15

S

Vorname	Nachname
Max	Meier

Alle Personen, die sowohl ein Buch
mit Preis 15 als auch einen Film mit
Preis 15 gekauft haben.

Relationale Algebra – Umbenennung

Kunde

R

K_ID	Name	Vorname	Geb. Datum
1	Meier	Moritz	01.01.2000
2	Müller	Max	01.01.1990
3	Mustermann	Mark	01.01.1950

Umbenennung von Attributen
und Tabellennamen

Kundeninformation

R

Kundennummer	Nachname	Vorname	Geb. Datum
1	Meier	Moritz	01.01.2000
2	Müller	Max	01.01.1990
3	Mustermann	Mark	01.01.1950

Entity Relationship Modell / Diagramm

ERM / ERD

Entity Relationship Modell/Diagramm – Begriffe

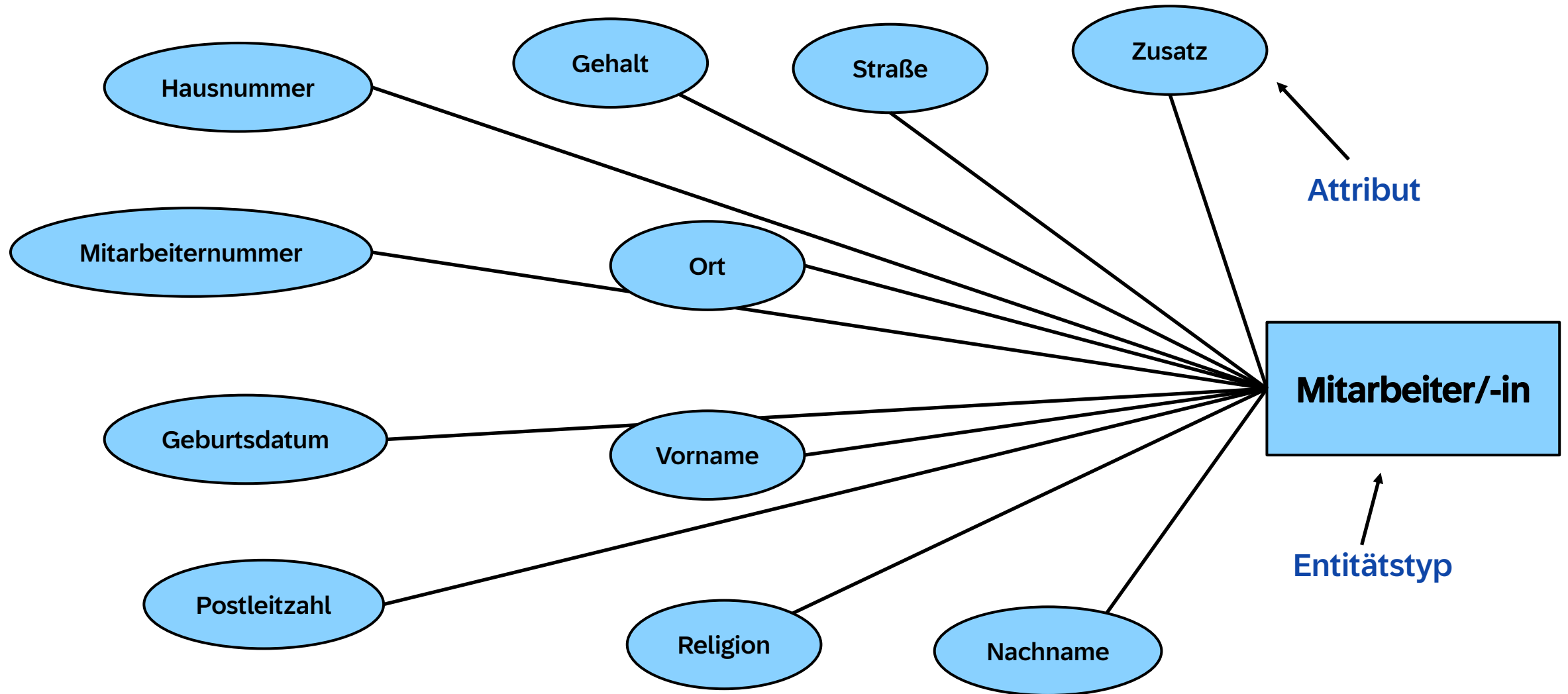
Begriff	Erläuterung
Entitätstyp (Kopfzeile)	Themenkreis welcher Elemente mit gleichen Merkmalen umfasst z.B. Person / Kurs
Entitätsmenge (Datensätze)	Zusammenfassung von Entitäten mit gleichen Eigenschaften unter einem eindeutigen gemeinsamen Oberbegriff
Entität (Datensatz)	Vollständiger Datensatz, ergeben die Entitätsmenge
Attribut (Spaltenname)	Merkmal / Eigenschaft eines Datensatzes in einer Entitätsmenge
Attributwert (Wert, Zelle)	Datenwert für zugehöriges Attribut

Darstellungen im Entity-Relationship-Diagramm – ERD

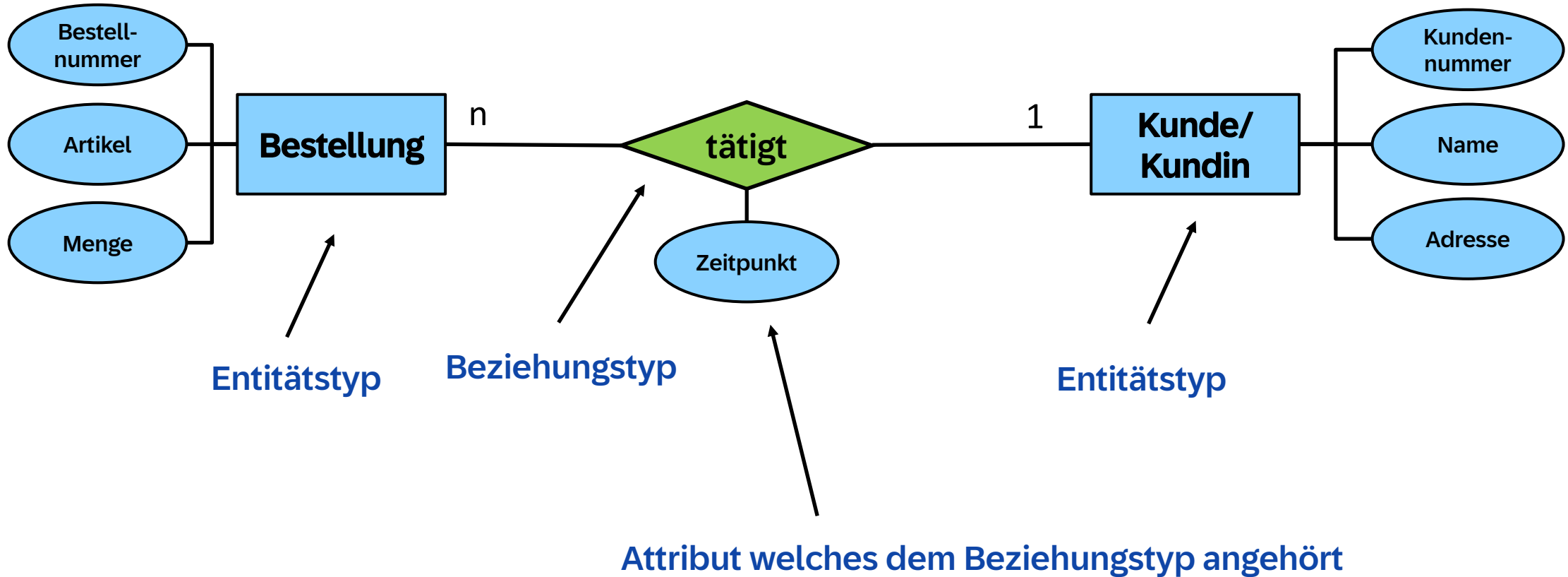


Beziehung → eine dedizierte Ausprägung einer Entität des Beziehungstypen

ERD – Entitätstyp

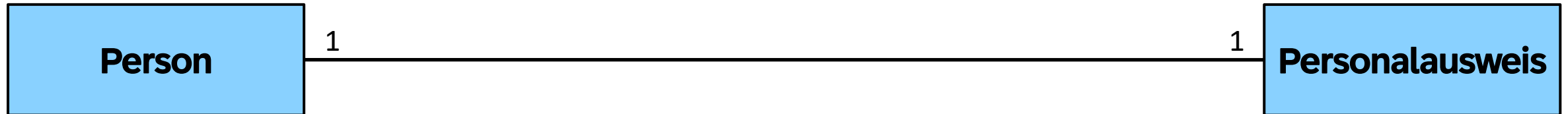


ERD – Beziehungstyp

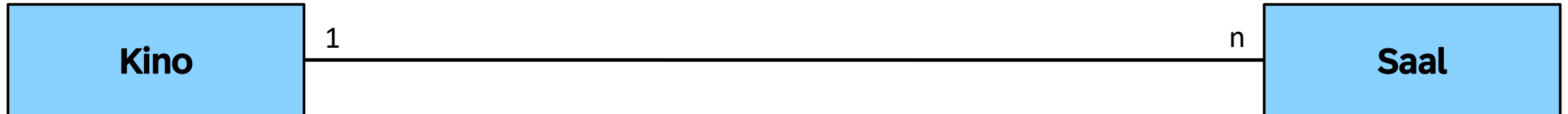


ERD – Kardinalität (Chen Notation)

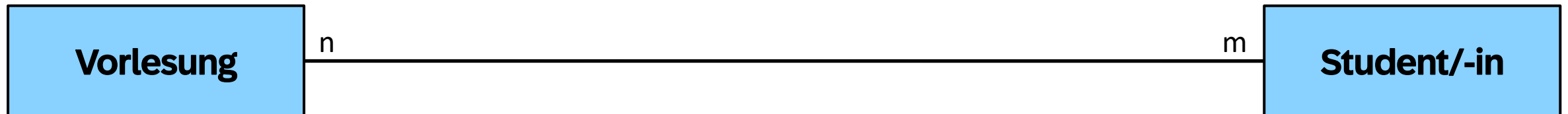
Eins zu eins – Eine Person besitzt eine Personalausweis und ein Personalausweis gehört einer Person



Eins zu viele – Ein Kino besitzt mehrere Säle und ein Saal gehört zu genau einem Kino

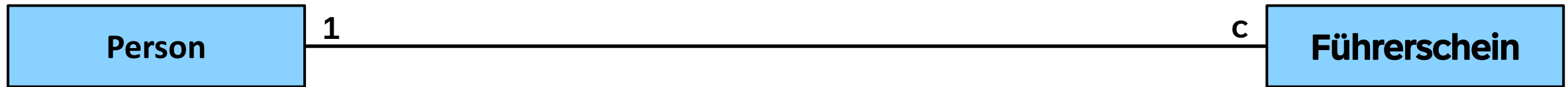


Viele zu viele – Eine Vorlesung hat viele Studierende und ein Studierender kann mehrere Vorlesungen besuchen

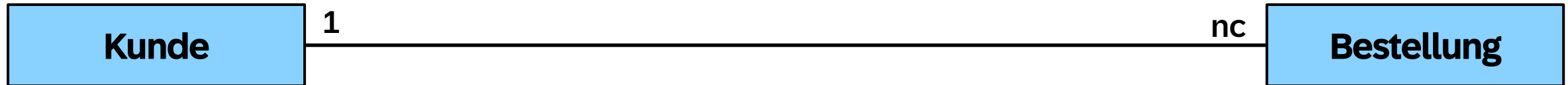


ERD – Kardinalität (Modifizierte Chen Notation)

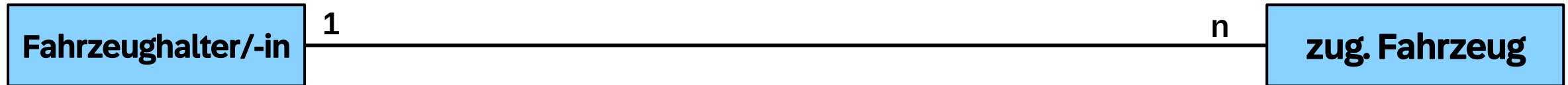
Eins zu eins – Eine Person besitzt einen oder keinen Führerschein und ein Führerschein gehört genau einer Person



Eins zu keinem bis viele – Ein Kunde kann beliebig viele Bestellungen aufgeben und eine Bestellung gehört zu genau einem Kunden



Mind. eins zu genau einem – Ein Fahrzeughalter muss mind. ein Fahrzeug besitzen und ein zugelassenes Fahrzeug gehört genau einem Fahrzeughalter

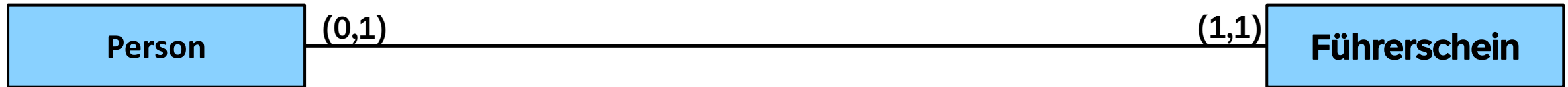


Jede Kardinalität lässt sich mit jeder anderen verknüpfen, möglich sind also auch die Kardinalitäten:

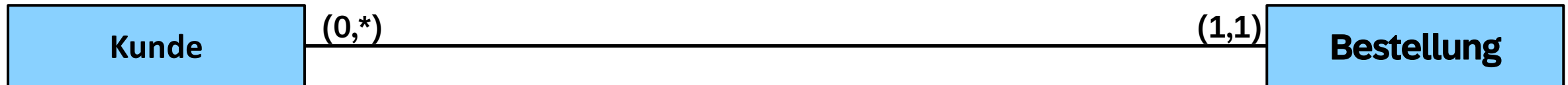
1:1 – c:c – c:n – c:nc – n:m – n:mc – nc:mc → natürlich auch in umgekehrter Reihenfolge

ERD – Kardinalität (Min-Max-Notation)

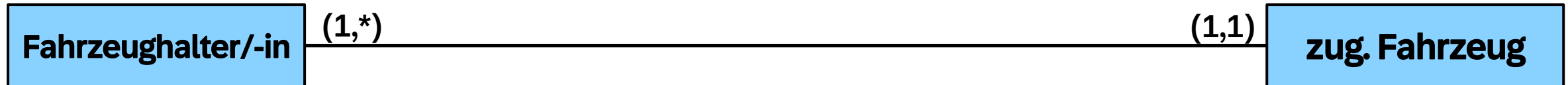
Eins zu eins – Eine Person besitzt einen oder keinen Führerschein und ein Führerschein gehört genau einer Person



Eins zu keinem bis viele – Ein Kunde kann beliebig viele Bestellungen aufgeben und eine Bestellung gehört zu genau einem Kunden



Mind. eins zu genau einem – Ein Fahrzeughalter muss mind. ein Fahrzeug besitzen und ein zugelassenes Fahrzeug gehört genau einem Fahrzeughalter

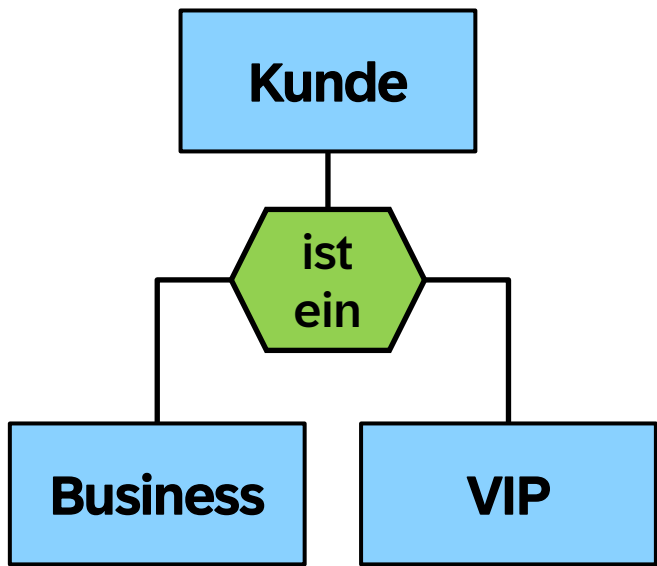


Jede Kardinalität lässt sich mit jeder anderen verknüpfen, möglich sind also auch die Kardinalitäten:

(1,1) : (1,1) – (0,1) : (0,1) – (0,1) : (1,*) – (0,1) : (0,*) – (1,*) : (1,*) – (1,*) : (0,*) – (0,*) : (0,*)
→ natürlich auch in umgekehrter Reihenfolge („*“ kann auch durch „n“ dargestellt werden)

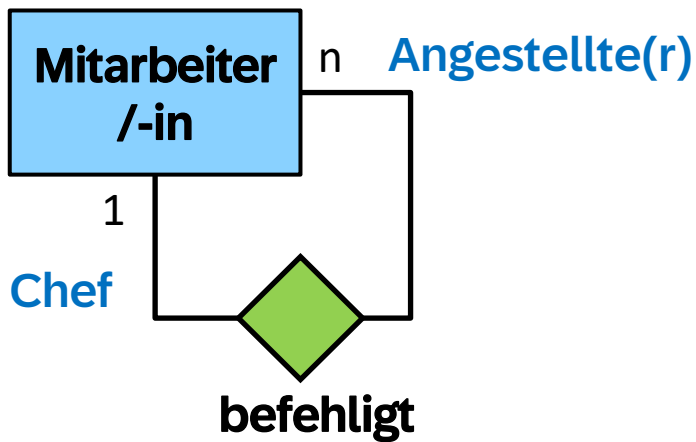
ERD – Spezialfälle

Generalisierung

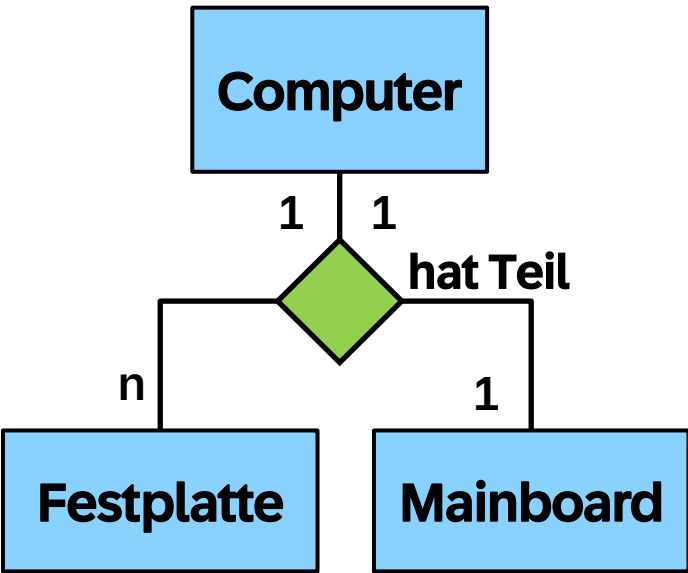


spezieller Beziehungstyp, Attribute werden vererbt und weitergegeben

Rekursion und Rollen

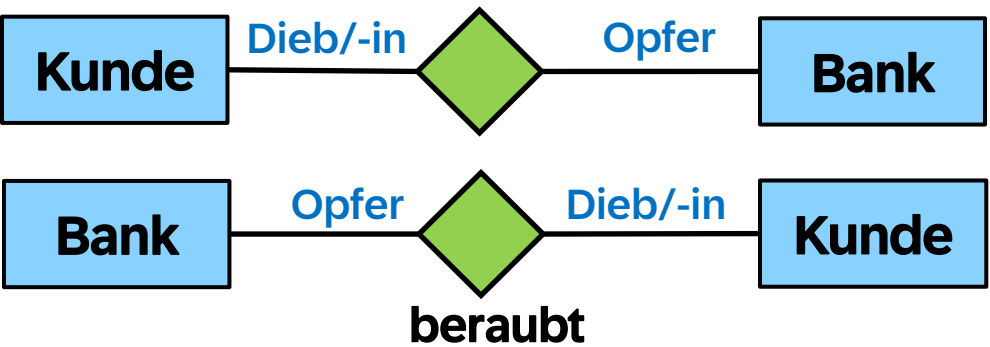


Komposition



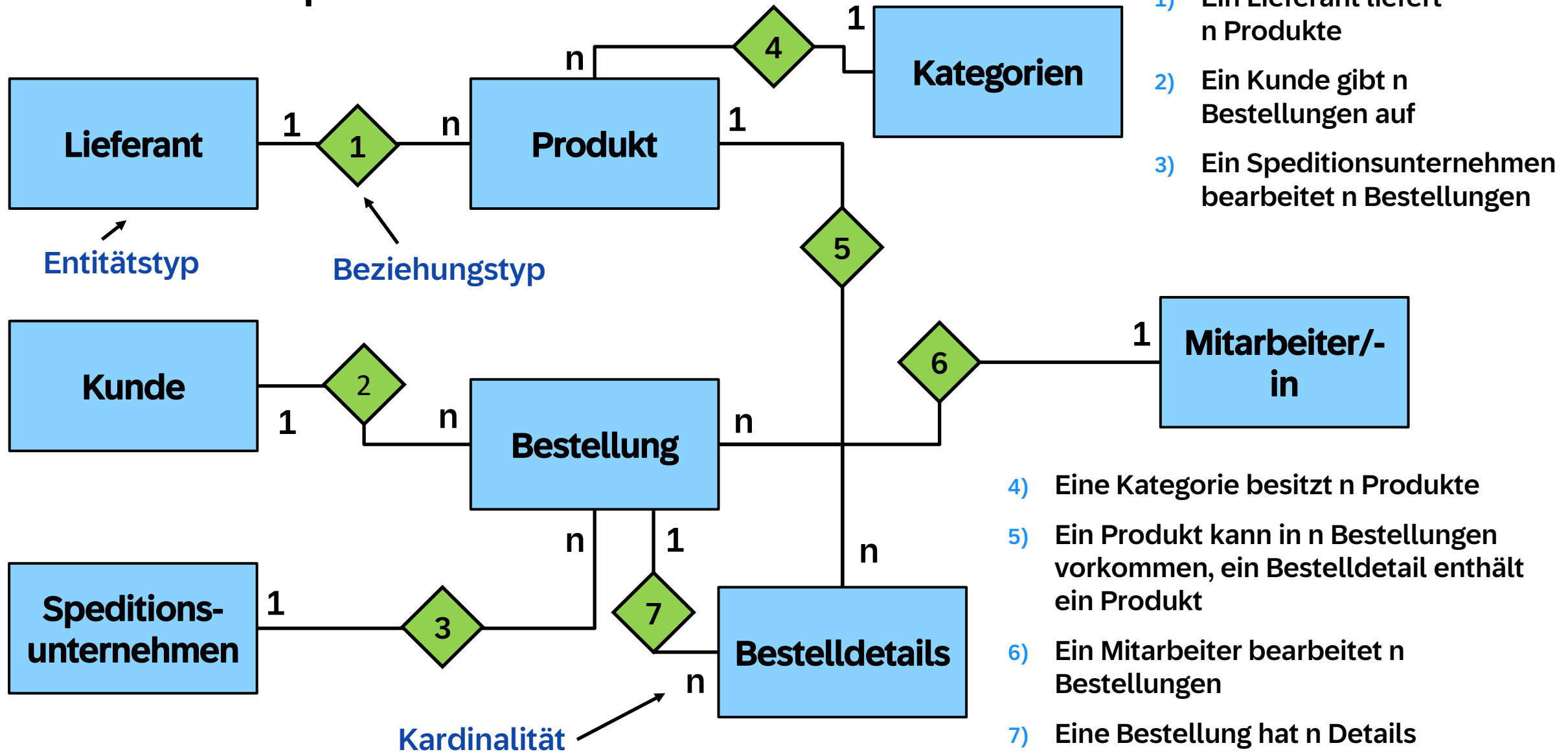
Normaler Beziehungstyp mit spezieller Bezeichnung

Leserichtung und Rollen



Normalerweise von links nach rechts, aber nicht immer möglich – zum besseren Verständnis Rollen ergänzen

ERD – Beispiel



Aufgabe 01

Aufgabe 01

Ein Freizeitpark beauftragt Sie, ein Datenmodell für die Verwaltung der Attraktionen, Besucher und Mitarbeiter zu erstellen. In Gesprächen mit dem Kunden haben sich folgende Anforderungen herausgestellt.

Der Freizeitpark besitzt verschiedene Arten von Attraktionen, nämlich Fahrgeschäfte und Shows. Fahrgeschäfte und Shows unterteilen sich jeweils nochmal in verschiedene Typen, diese haben aber keine zusätzlichen Eigenschaften bis auf eine entsprechende Beschreibung. **Achterbahnen, Wasserbahnen, Karussell, Schiffschaukel und Eisenbahn sind Fahrgeschäfte.** Es gibt **Stunt-, Eiskunstlauf- und Unterhaltungsshow.** **Attraktionen werden von Mitarbeitern betreut.** **Ein Mitarbeiter kann mehrere Attraktionen betreuen, je nach Attraktion soll die Möglichkeit bestehen ihr mehrere Mitarbeiter zuzuweisen.** **Attraktionen können von Besuchern besucht werden, jeder Besucher kann zu einem Zeitpunkt eine Attraktion besuchen, aber insgesamt (z.B. über den Tag verteilt) mehrere Attraktionen.** Ebenso ist es möglich, dass **eine Attraktion von mehreren Besuchern gleichzeitig besucht werden kann.** **Besucher zahlen einmalig Eintritt beim Betreten des Freizeitparks,** alle Attraktionen innerhalb des Parks sind dann kostenfrei, diese Eintrittspreise sollen ebenfalls festgehalten und in das Datenmodell integriert werden. Die Eigenschaften finden Sie auf der nächsten Folie.

Erstellen Sie ein ERD (Entitäten, Beziehungen, Kardinalitäten und Attribute) zum Abbilden der Anforderungen!

Aufgabe 01

Besucher

- Geburtsdatum
- Vorname
- Nachname
- Budget

Attraktion

- Bezeichnung
- Öffnungszeit
- Schließzeit

Eintritt

- Kategorie
- Eintritt
- Maximales Alter

Mitarbeiter

- Vorname
- Nachname
- Geburtsdatum
- Gehalt (darf auf Grund von Tarifverträgen nicht kleiner als 1250€ sein)
- Adresse
- Telefonnummer

Show

- Dauer
- Art der Show (z.B. Eiskunstlauf, Stunt)

Fahrgeschäft

- Höchstgeschwindigkeit (in km/h)
- Fahrzeit (Zeit auf dem Fahrgeschäft inkl. z.B. Warten zum Start und Aussteigen)
- Maximale G-Kräfte
- Mindestgröße
- Mindestalter
- Hinweis
- Geeignet für Schwangere
- Art des Fahrgeschäftes (z.B. Achterbahn, Wasserbahn)

Relationenmodell

Relationenmodell – Edgar Frank Codd

- **britischer Informatiker (1923 – 2003), gilt als Vater des relationalen Datenbankmodells**
- **Forscher bei IBM → Entwicklung theoretischer Grundlagen für relationale Datenbanken (1960er/1970er Jahre)**
- **1970 – Veröffentlichung Paper „A Relational Model of Data for Large Shared Data Banks“ → Vorstellung relationales Modell**
- **Definition relationale Algebra**
- **Entwicklung System R (Prototyp relationales Datenbankmanagementsystem), Abfragesprache SEQUEL, Basis für SQL**

Relationenmodell – Begriffe

Relationsschema

→ Menge von Attributen (hier Studierendenschema)

Schema

→ Gesamtstruktur der Tabellen, Attribute, Beziehungen

Relation

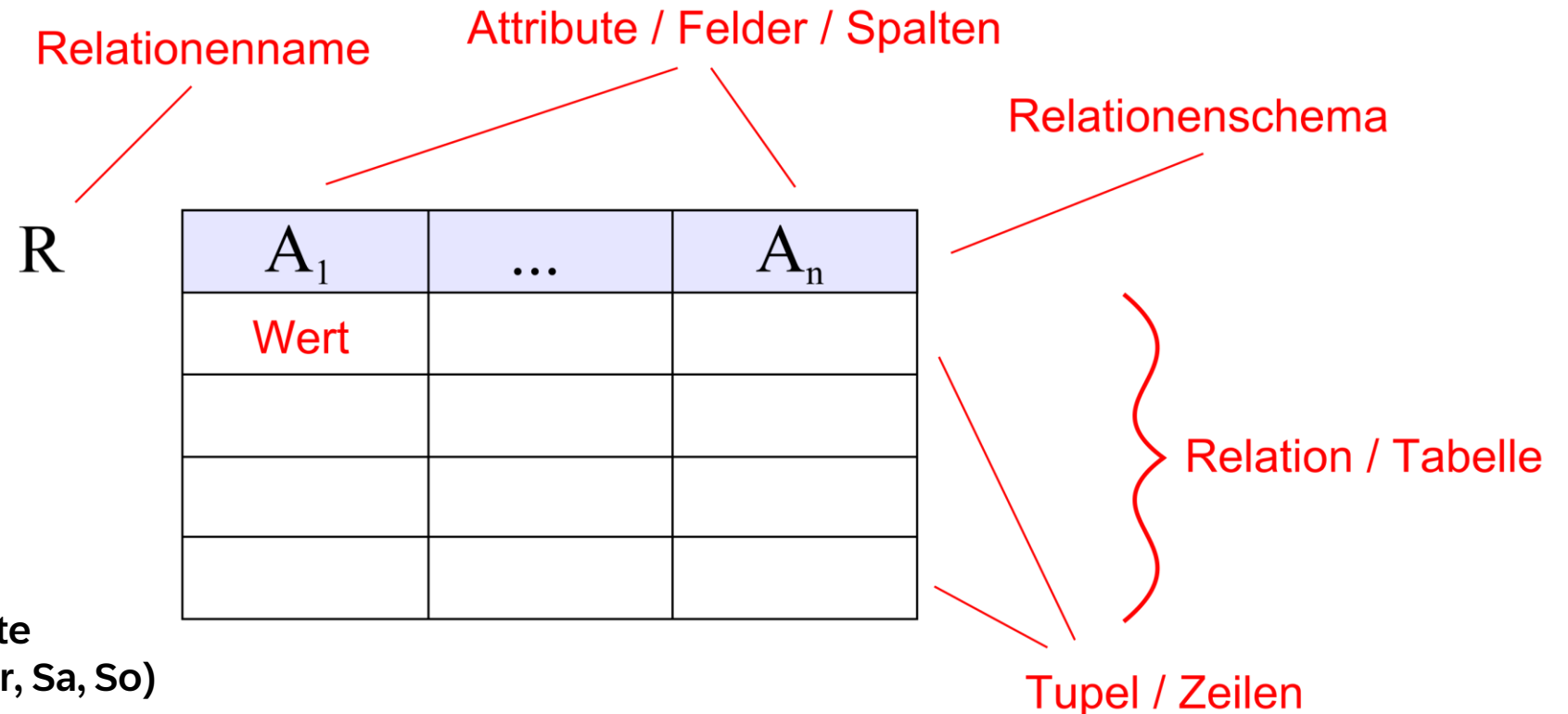
→ Ausprägung eines Relationsschemas (Tabelle)

(hier z.B. Bachelor Studierende / Master Studierende)

→ Beziehung zwischen Elementen

Domäne (Wertebereich)

→ Einschränkungen der Attributwerte
(z.B. Wochentag nur Mo, Di, Mi, Do, Fr, Sa, So)



Relationenmodell – Schlüssel

Superschlüssel

- Attribute die einen Datensatz eindeutig identifizieren
- z.B. Mitarbeiter/-in mit Attribut „Name“, „Vorname“, „Geburtsdatum“ und „Personalnummer“, beides wären mögliche Superschlüssel

Kandidatschlüssel

- Superschlüssel mit minimaler Menge Schlüsselattributen

Primärschlüssel

- einer der Kandidatschlüssel wird zum Primärschlüssel, übrige Kandidatschlüssel nennt man „**Alternativschlüssel**“
- eindeutige Identifikation einer Zeile, möglichst wenig Attribute, keine Änderung
- kann aus einem oder einer Kombination mehrerer Attribute bestehen („**zusammengesetzter Schlüssel**“ / „**Composite Key**“)
- im ERD unterstrichen dargestellt wenn angegeben, andernfalls erst bei Überführung ermittelt

Fremdschlüssel

- Attribut oder Kombination aus Attributen welches in anderer Relation Primärschlüssel ist

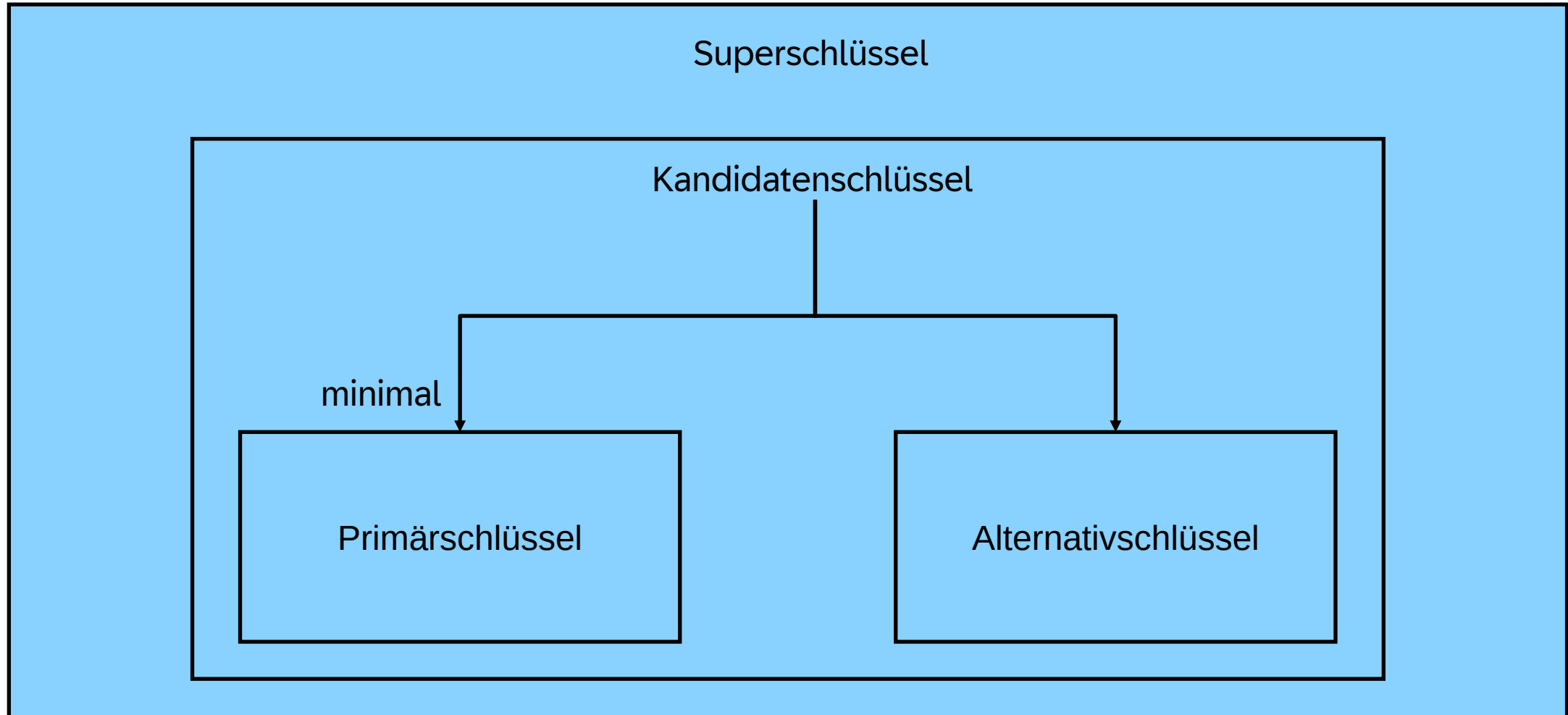
Schlüsselattribut

- Attribut, das Bestandteil eines Kandidatschlüssels ist

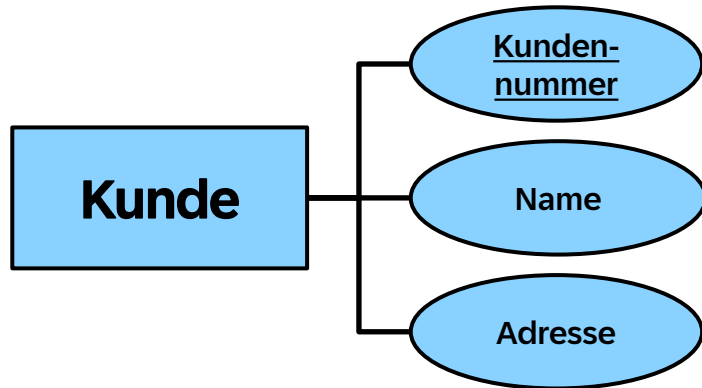
Nichtschlüsselattribut

- Attribut, das kein Bestandteil eines Kandidatschlüssels ist

Relationenmodell – Schlüssel

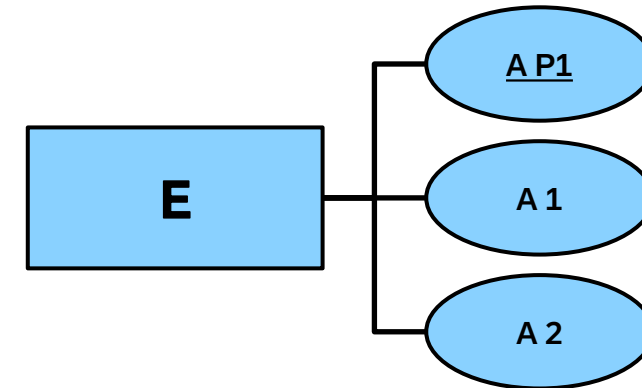


Relationenmodell – Entitätstyp



Relationsschema

Kunden(K_ID: int, Name: string, Adresse: string)



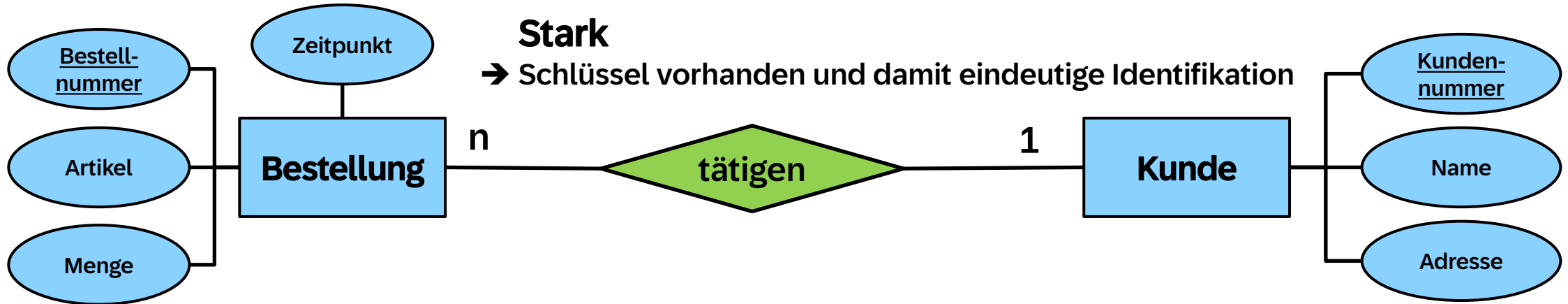
Relationsschema

E(A P1: Typ P1, A 1: Typ 1, A 2 : Typ 2, ...)

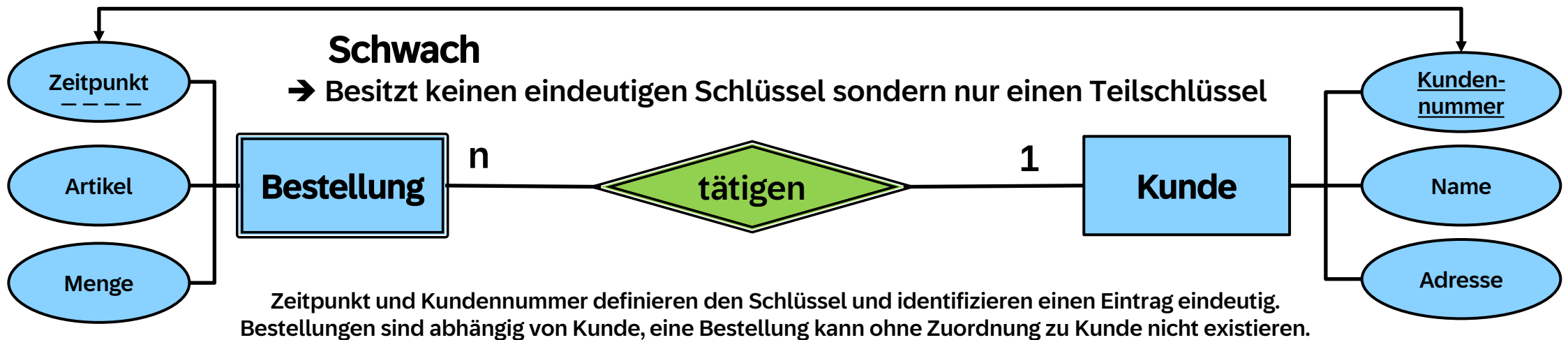
Kunde ist eine Ausprägung des Relationsschemas Kunden, Lieblingskunden wäre ebenfalls eine Ausprägung des Relationsschemas Kunden.

Jeder Entitätstyp wird bei der Überführung ins Relationenmodell zu einem Relationsschema.

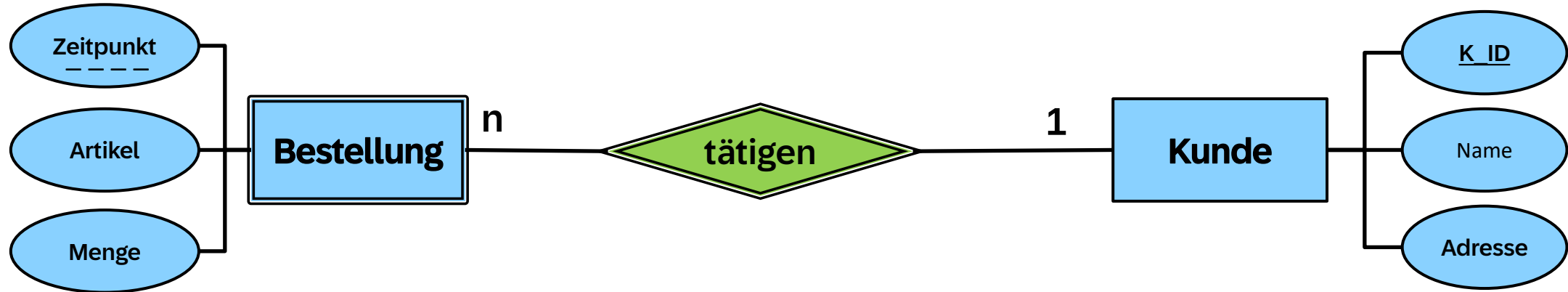
Relationenmodell – Starker & schwacher Entitätstyp



Ist eine Kardinalität mit 1 (muss) definiert kann man den Entitätstyp auch als schwach definieren)



Relationenmodell – Schwacher Entitätstyp

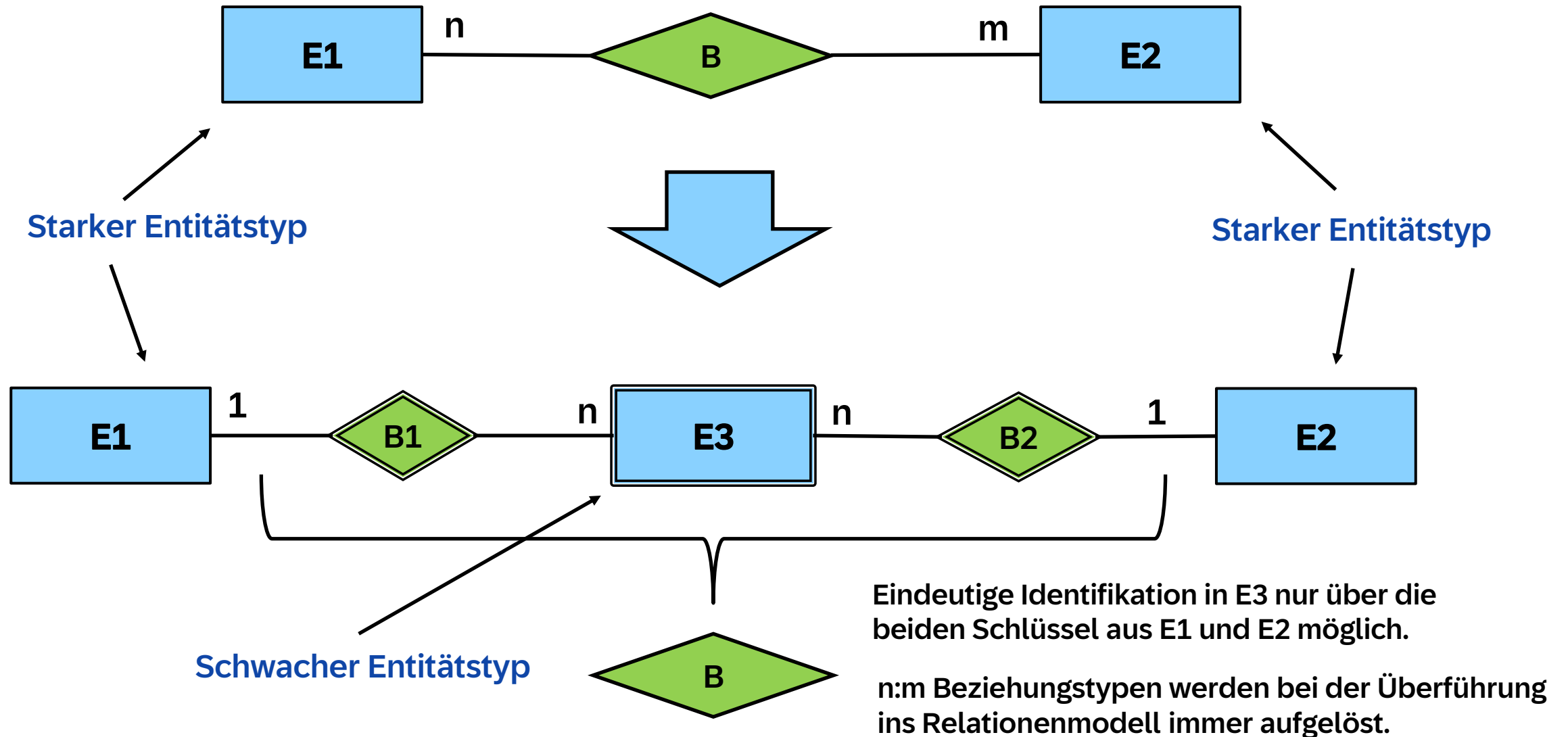


Durch den schwachen Entitätstypen kann für den Beziehungstypen kein Relationsschema angelegt werden, da kein Schlüssel für den schwachen Entitätstypen vorhanden ist.

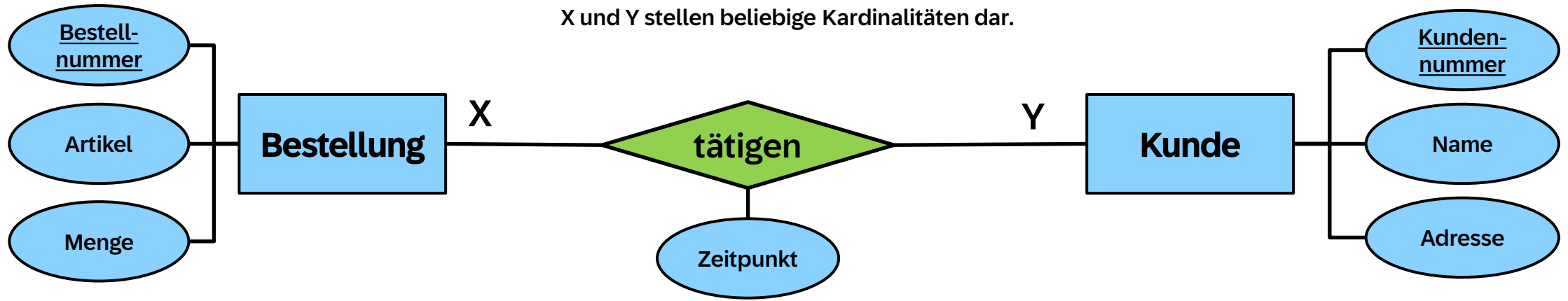
Kunden(K_ID: int, Name: string, Adresse: string)

Bestellungen(K_ID: int, Zeitpunkt: Timestamp, Artikel: string, Menge: int)

Relationenmodell – Schwache Entitätstypen bei n:m



Relationenmodell – Binäre Beziehungstypen



$X \rightarrow 1 / Y \rightarrow n$

`tätigen(K_ID: int, B_ID: int, Zeitpunkt: Timestamp)`

$X \rightarrow n / Y \rightarrow 1$

`tätigen(K_ID: int, B_ID: int, Zeitpunkt: Timestamp)`

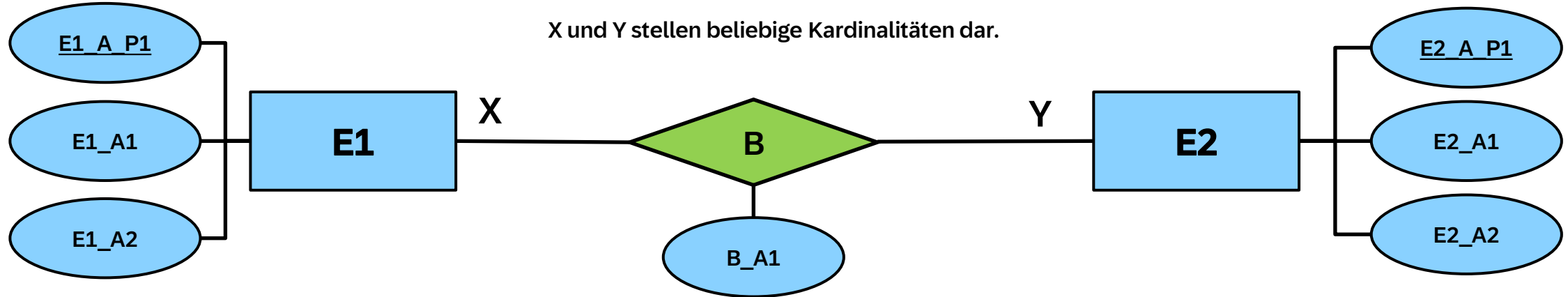
$X \rightarrow n / Y \rightarrow m$

`tätigen(K_ID: int, B_ID: int, Zeitpunkt: Timestamp)`

Schlüssel sind
abhängig von der
Kardinalität!

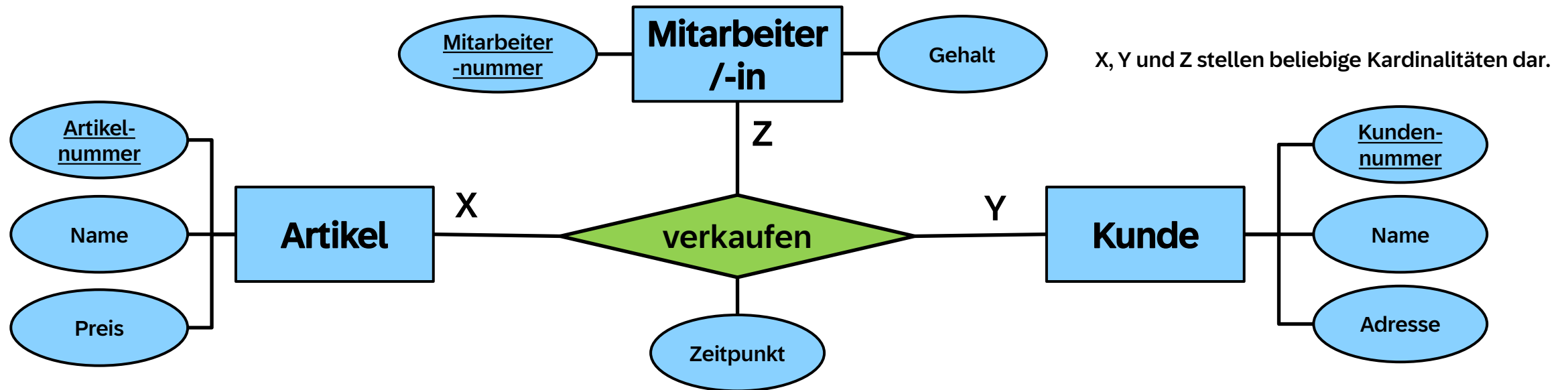
Jeder Beziehungstyp (bei starken Entitätstypen) wird zunächst zu einem Relationsschema. Je nach Kardinalität kann dies später vereinfacht werden.

Relationenmodell – Binäre Beziehungstypen



B(E1_A_P1: Typ, E2_A_P1: Typ, B_A1: Typ, ...)

Relationenmodell – Mehrstellige Beziehungstypen



verkaufen(K_ID: int, A_ID: int, M_ID: int, Zeitpunkt: Timestamp)

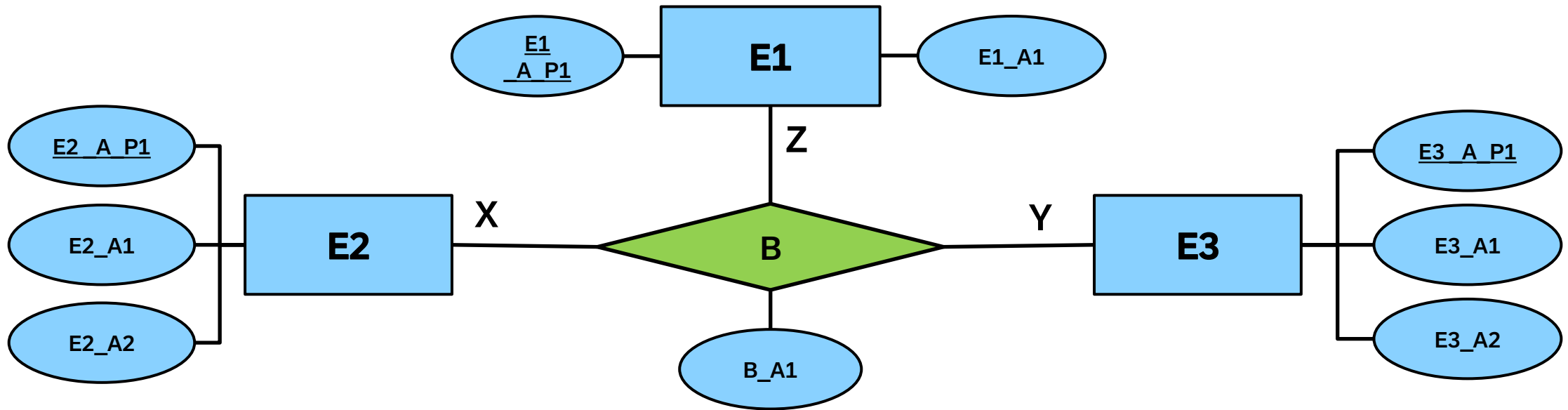
$X \rightarrow n / Y \rightarrow m / Z \rightarrow 1$

Kunde und Artikel bestimmen Mitarbeiter

verkaufen(K_ID: int, A_ID: int, *M_ID*: int, Zeitpunkt: Timestamp)

Schlüssel sind abhängig von der Kardinalität!

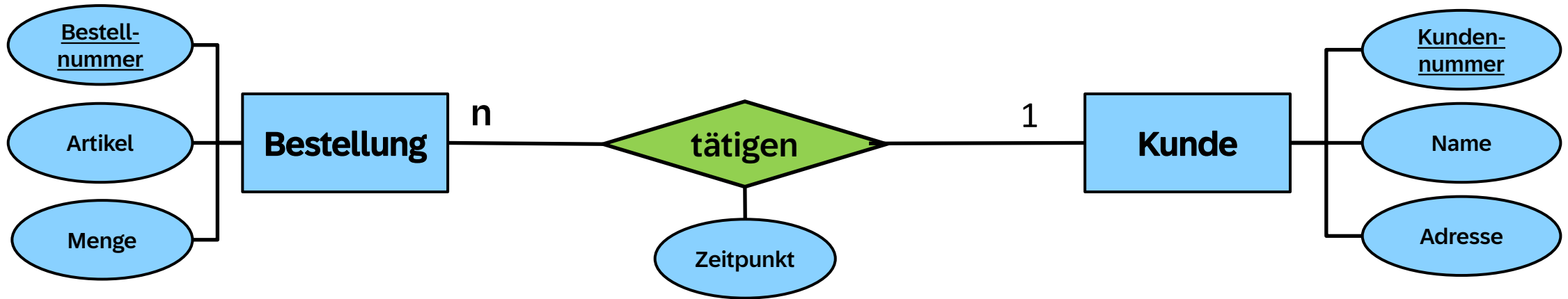
Relationenmodell – Mehrstellige Beziehungstypen



X, Y und Z stellen beliebige Kardinalitäten dar.

$B(E1_A_P1: \text{Typ}, E2_A_P1: \text{Typ}, E3_A_P1: \text{Typ}, B_A1: \text{Typ}, \dots)$

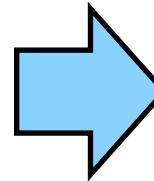
Relationenmodell – Vereinfachung Primärschlüssel



Kunden(K_ID: int, Name: string, Adresse: string)

tätigen(K_ID: int, B_ID: int, Zeitpunkt: Timestamp)

Bestellungen(B_ID: int, Artikel: string, Menge: int)

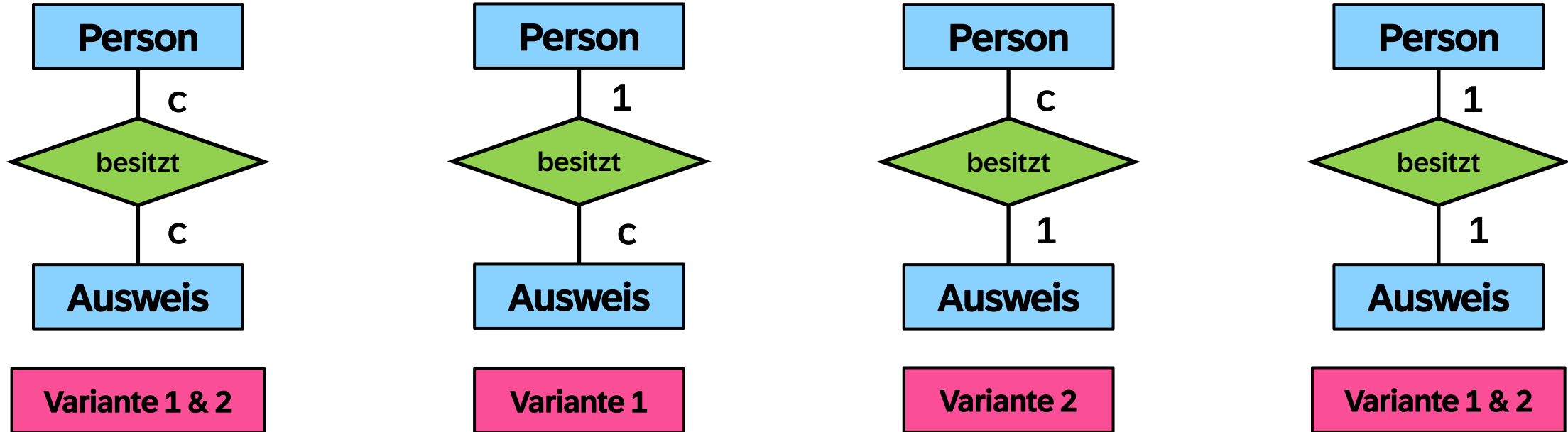


Kunden(K_ID: int, Name: string, Adresse: string)

Bestellungen(B_ID: int, Artikel: string, Menge: int, Zeitpunkt: Timestamp, K_ID: int)

Relationsschemata mit gleichem Primärschlüssel können zusammengefasst werden.

Relationenmodell – Vermeidung fehlender Fremdschlüssel



Variante 1

Person(P_ID: int, Name: string, Adresse: string)

Ausweis(Ausweisnummer: int, Ausstelldatum: Date, P_ID: int)

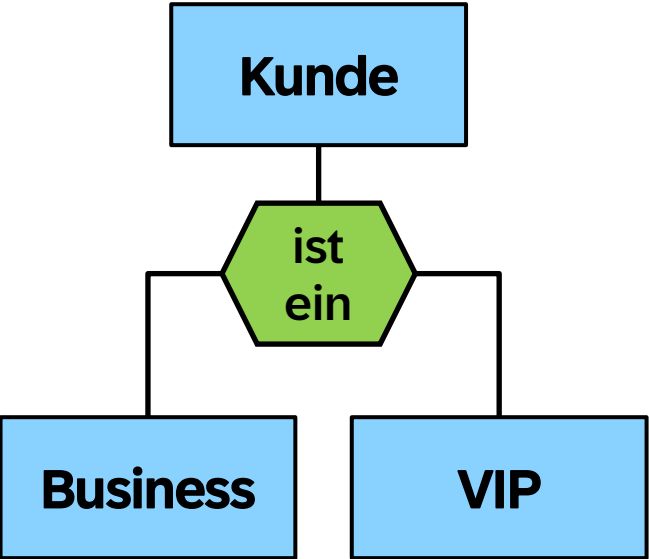
Variante 2

Person(P_ID: int, Name: string, Adresse: string, Ausweisnummer: int)

Ausweis(Ausweisnummer: int, Ausstelldatum: Date)

Nur für Kardinalität 1:1 anzuwenden. Ziel ist es auf der Datenbank später NULL Werte zu vermeiden.

Relationenmodell – Generalisierung (vertikale Partitionierung)



Kunde

K_ID	Name	Vorname	Geb. Datum
1	Meier	Moritz	01.01.2000
2	Müller	Max	01.01.1990
3	Mustermann	Mark	01.01.1950

Busines

K_ID	Rabatt (%)
1	10

VIP

K_ID	Bonuspunkte
2	2500

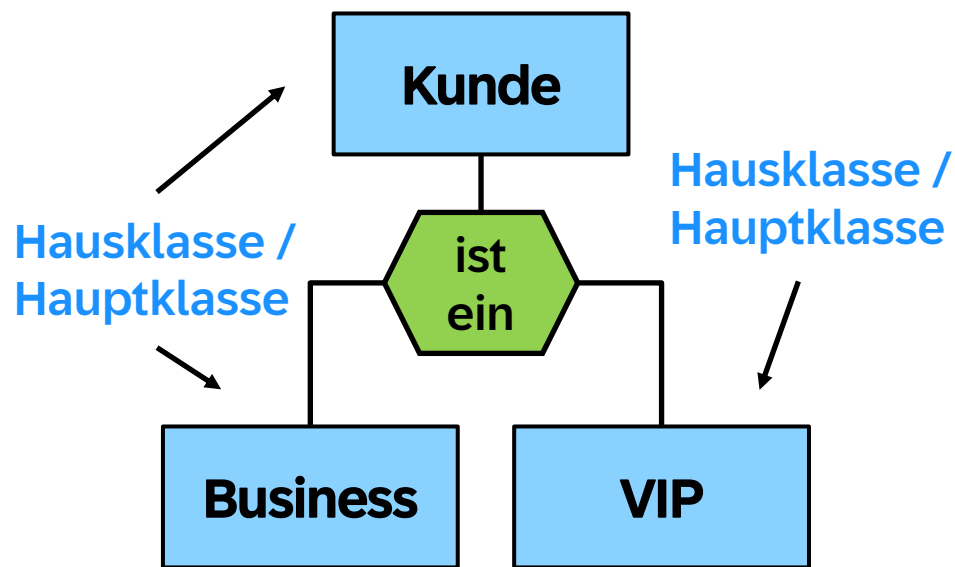
Kunden(K_ID: int, Name: string, Vorname: string, Geburtsdatum: Date)

Business(K_ID: int, Rabatt: float)

VIP(K_ID: int, Bonuspunkte: int)



Relationenmodell – Generalisierung (horizontale Partitionierung)



Kunden(K_ID: int, Name: string,
Vorname: string, Geburtsdatum: Date)



Business(K_ID: int, Name: string, Vorname: string,
Geburtsdatum: Date, Rabatt: float)

VIP(K_ID: int, Name: string, Vorname: string,
Geburtsdatum: Date, Bonuspunkte: int)

Ohne Tupel Redundanz (jedes Tupel
nur in Hausklasse gespeichert)

Kunde

K_ID	Name	Vorname	Geb. Datum
3	Mustermann	Mark	01.01.1950

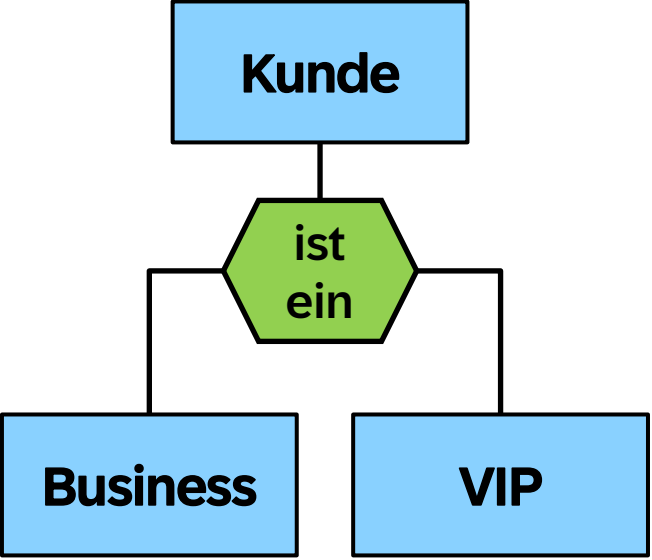
Busines

K_ID	Name	Vorname	Geb. Datum	Rabatt (%)
1	Meier	Moritz	01.01.2000	10

VIP

K_ID	Name	Vorname	Geb. Datum	Bonuspunkte
2	Müller	Max	01.01.1990	2500

Relationenmodell – Generalisierung (horizontale Partitionierung)



Mit Tupel Redundanz (jedes Tupel der Hauptrelation ist in jeder Relation nochmal vorhanden)

Kunde

K_ID	Name	Vorname	Geb. Datum
1	Meier	Moritz	01.01.2000
2	Müller	Max	01.01.1990
3	Mustermann	Mark	01.01.1950

Business

K_ID	Name	Vorname	Geb. Datum	Rabatt (%)
1	Meier	Moritz	01.01.2000	10

VIP

K_ID	Name	Vorname	Geb. Datum	Bonuspunkte
2	Müller	Max	01.01.1990	2500

Kunden(K_ID: int, Name: string, Vorname: string, Geburtsdatum: Date)



Business(K_ID: int, Name: string, Vorname: string, Geburtsdatum: Date, Rabatt: float)

VIP(K_ID: int, Name: string, Vorname: string, Geburtsdatum: Date, Bonuspunkte: int)

Aufgabe 02

Aufgabe 02

Überführen Sie das bereits erstellte ERD in das Relationenmodell.

Normalisierung

Normalisierung – Funktionale Abhängigkeit

Eine funktionale Abhängigkeit zwischen Attribut Y und Attribut X liegt dann vor, wenn es zu jedem X genau ein Y gibt.

<u>Kundennummer</u>	Vorname	Name	Postleitzahl	Stadt
1	Max	Mustermann	68161	Mannheim
2	Marie	Musterfrau	69115	Heidelberg

Name ist funktional abhängig von Kundennummer.

Vorname ist funktional abhängig von Kundennummer.

Postleitzahl ist funktional abhängig von Kundennummer.

Kundennummer $\rightarrow$ Name

Kundennummer $\rightarrow$ Vorname

Kundennummer $\rightarrow$ Postleitzahl

Triviale funktionale Abhängigkeit

$\rightarrow$ Attributmenge ist Teilmenge der Abhängigkeit der rechten Seite

$\rightarrow$ Y ist Teilmenge von X

Allgemein: $X \rightarrow Y$

Kundennummer, Name $\rightarrow$ Name

$\rightarrow$ Triviale funktionale Abhängigkeit

Normalisierung – Voll funktionale Abhängigkeit

Eine voll funktionale Abhängigkeit liegt dann vor, wenn das Nicht-Schlüsselattribut nicht nur von einem Teil der Attribute eines zusammengesetzten Schlüsselkandidaten funktional abhängig ist.

→ Nicht-Schlüsselattribut muss von allen Attributen zusammengesetzter Schlüsselkandidaten abhängen

Kundennummer	<u>Bestellnummer</u>	<u>Artikelnummer</u>	Menge	Lieferadresse
1	12345	5	2	Adresse 01
1	12345	8	3	Adresse 02

Menge ist vollfunktional abhängig von der Attributgruppe Bestellnummer und Artikelnummer.

Lieferadresse ist vollfunktional abhängig von der Attributgruppe Bestellnummer und Artikelnummer.

Bestellnummer, Artikelnummer → Menge

Bestellnummer, Artikelnummer → Lieferadresse

Normalisierung – Transitive Abhängigkeit

Eine transitive Abhängigkeit liegt dann vor, wenn Y von X funktional abhängig und Z von Y, so ist Z von X funktional abhängig. Diese Abhängigkeit ist transitiv.

<u>Kundennummer</u>	Vorname	Name	Postleitzahl	Stadt
1	Max	Mustermann	68161	Mannheim
2	Marie	Musterfrau	69115	Heidelberg

Name ist funktional abhängig von Kundennummer.

Vorname ist funktional abhängig von Kundennummer.

Postleitzahl ist funktional abhängig von Kundennummer.

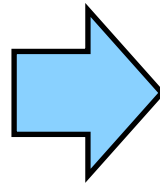
Stadt ist funktional abhängig von Postleitzahl.

Y = Postleitzahl / X = Kundennummer / Z = Stadt

$X \rightarrow Y$

$Y \rightarrow Z$

$X \rightarrow Z$

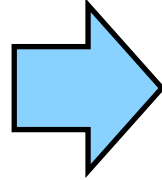


Daraus ergibt sich, dass Stadt transitiv abhängig von Kundennummer ist.

Normalisierung – 1. Normalform (1NF)

Ein Relationsschema befindet sich in der ersten Normalform, wenn die Wertebereiche der Attribute des Relationsschema atomar sind.

<u>Person</u>	Führerscheinklassen
1	A, A1, C
2	A, A1, B
3	B, BE



<u>Person</u>	<u>Führerscheinklasse</u>
1	A
1	A1
1	C
2	A
2	A1
2	B
3	B
3	BE

Ist durch das Relationenmodell oft bereits gegeben.

Normalisierung – 2. Normalform (2NF)

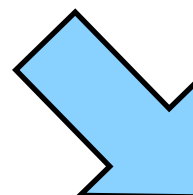
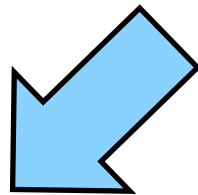
Ein Relationsschema befindet sich genau dann in der zweiten Normalform, wenn er sich in der ersten Normalform befindet und jedes Nichtschlüsselattribut von jedem Schlüsselkandidaten voll funktional abhängig ist.

Jedes nicht-primäre Attribut (nicht Teil eines Schlüssels) ist jeweils von allen ganzen Schlüsseln abhängig, nicht nur von einem Teil eines Schlüssels. Wichtig ist hierbei, dass die Nichtschlüsselattribute wirklich von allen Schlüsseln vollständig abhängen. Somit gilt, dass Relationsschemata in der 1NF, deren Schlüsselkandidat(en) nicht zusammengesetzt sind, sondern lediglich aus jeweils (einem) einzelnen Attribut(en) bestehen, automatisch die 2NF erfüllen.

Kunden-nummer	<u>Bestell-nummer</u>	Datum	<u>Artikel-nummer</u>	Menge	Lieferadresse
1	12345	01.01.2020	5	2	Adresse 01
1	12345	01.01.2020	8	3	Adresse 02

voll funktionale Abhängigkeiten

Bestellnummer, Artikelnummer → Menge
Bestellnummer, Artikelnummer → Lieferadresse
Bestellnummer → Kundennummer
Bestellnummer → Datum



Kundennummer	<u>Bestellnummer</u>	Datum
1	12345	01.01.2020

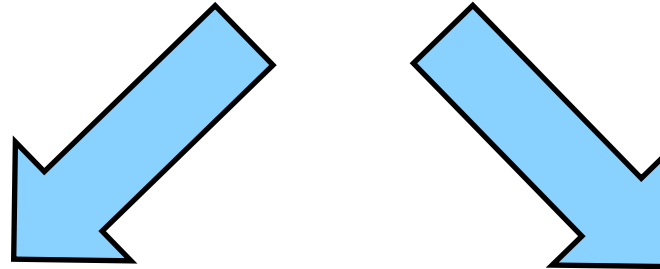
<u>Bestellnummer</u>	<u>Artikelnummer</u>	Menge	Lieferadresse
12345	5	2	Adresse 01
12345	8	3	Adresse 02

Normalisierung – 3. Normalform (3NF)

Ein Relationenschema befindet sich genau dann in der dritten Normalform, wenn es sich in der zweiten Normalform befindet und kein Nichtschlüsselattribut transitiv von einem Schlüsselkandidaten abhängt.

<u>Kundennummer</u>	Vorname	Name	Postleitzahl	Stadt
1	Max	Mustermann	68161	Mannheim
2	Marie	Musterfrau	69115	Heidelberg

Stadt ist transitiv abhängig von Kundennummer



<u>Kundennummer</u>	Vorname	Name	Postleitzahl
1	Max	Mustermann	68161
2	Marie	Musterfrau	69115

<u>Postleitzahl</u>	Stadt
68161	Mannheim
69115	Heidelberg

Normalisierung – Boyce-Codd Normalform (3.5NF/BCNF)

Ein Relationenschema befindet sich genau dann in der Boyce-Codd Normalform, wenn es sich in der dritten Normalform befindet und jede Determinante ein Schlüsselkandidat oder die Abhängigkeit trivial ist.

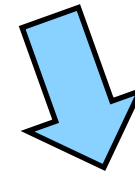
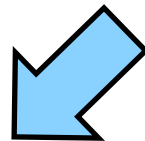
Tritt nur bei mehreren Schlüsselkandidaten mit überlappenden Attributen auf.

Mögliche Schlüsselkandidaten:

Matrikelnummer & Vorlesung / Matrikelnummer & Dozent

<u>Matrikelnummer</u>	<u>Vorlesung</u>	Dozent
123	Datenbanken	Nicola
456	Datenbanken	Nicola
123	Software Engineering	Meier

Jeder Dozent bietet genau eine Vorlesung an.



<u>Matrikelnummer</u>	<u>Vorlesung</u>
123	Datenbanken
456	Datenbanken
123	Software Engineering

<u>Vorlesung</u>	Dozent
Datenbanken	Nicola
Software Engineering	Meier

Determinante

→ Attributmenge, von der anderen Attributen voll funktional abhängen

Dozent → Vorlesung

Matrikelnummer, Dozent → Vorlesung

Dozent ist funktional abhängig von Vorlesung, damit ist Dozent eine Determinante – aber kein Schlüsselkandidat.

Normalisierung – Mehrwertige Abhängigkeit

Ein Wert von X kann mehrere Werte von Y bestimmen, unabhängig davon kann A auch Werte von anderen Attributen bestimmen.

<u>Person</u>	<u>Spiel</u>	<u>Buch</u>
Max	Monopoly	Percy Jackson
Max	Risiko	Percy Jackson
Christina	Monopoly	Herr der Ringe
Christina	Risiko	Herr der Ringe

X → Person
Y → Spiel
Z → Buch

Eine Person kann mehrere Spiele besitzen, unabhängig davon kann eine Person auch Bücher besitzen, Spiele und Bücher besitzen dabei keinerlei Abhängigkeit zueinander, Spiel hat also keinen Einfluss auf Buch und umgekehrt.

X →→ Y
X →→ Z

Nachteil → Neues Buch für Max:

<u>Person</u>	<u>Spiel</u>	<u>Buch</u>
Max	Monopoly	Herr der Ringe
Max	Risiko	Herr der Ringe

Normalisierung – 4. Normalform (4NF)

Ein Relationenschema befindet sich genau dann in der 4. Normalform, wenn es sich in der Boyce-Codd Normalform befindet und nur triviale mehrwertige Abhängigkeiten besitzt, die voneinander unabhängig sind.

Es darf als nicht mehrere „:-n“ Beziehungen in einer Relation geben, die inhaltlich nichts miteinander zu tun haben.

<u>Person</u>	<u>Spiel</u>	<u>Buch</u>
Max	Monopoly	Percy Jackson
Max	Risiko	Percy Jackson
Max	Monopoly	Herr der Ringe
Max	Risiko	Herr der Ringe
Christina	Monopoly	Herr der Ringe
Christina	Risiko	Herr der Ringe

Person → Spiel
Person → Buch

Trivial bedeutet auch, dass die Relation nur Attribute besitzt, die Teil der mehrwertigen Abhängigkeit sind.

<u>Person</u>	<u>Buch</u>
Max	Percy Jackson
Max	Herr der Ringe
Christina	Herr der Ringe

Triviale mehrwertige Abhängigkeit, da alle Attribute der Relation Teil der mehrwertigen Abhängigkeit sind.

<u>Person</u>	<u>Spiel</u>
Max	Monopoly
Max	Risiko
Christina	Monopoly
Christina	Risiko

Normalisierung – 5. Normalform (5NF)

Ein Relationenschema befindet sich genau dann in der 5. Normalform, wenn es sich in der 4. Normalform befindet und keine mehrwertigen Abhängigkeiten besitzt, die voneinander abhängig sind.

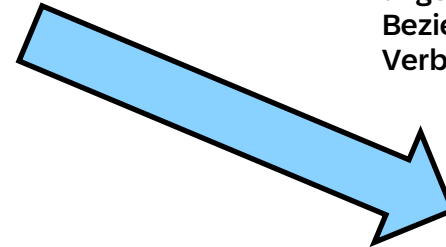
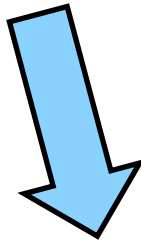
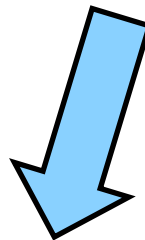
Es darf als nicht mehrere „:-n“ Beziehungen in einer Relation geben, unabhängig ob diese inhaltlich etwas miteinander zu tun haben oder nicht.

<u>Person</u>	<u>Konsole</u>	<u>Spiel</u>
Max	X-Box	Tomb Raider
Max	Playstation	Call of Duty
Marie	X-Box	Call of Duty

Person → Konsole

Person → Spiel

Konsole → Spiel



**Kombinieren der Daten führt
nachträglich zu neuen Datensätzen.**

Die 5. Normalform dient anders als die anderen Normalformen dazu, neue Informationen zu entdecken. Richtig angewendet ergeben sich neue Zusammenhänge in den Daten. Sie wird nur angewandt, wenn man mögliche Verbindungen aus mehreren Beziehungen ausdrücken möchte und keine konkreten Verbindungen zwischen den Tabellen abbildet.

Welche Person besitzt welche Konsole?

<u>Person</u>	<u>Konsole</u>
Max	X-Box
Max	Playstation
Marie	X-Box

Welche Konsole besitzt welche Spiele?

<u>Konsole</u>	<u>Spiel</u>
X-Box	Tomb Raider
Playstation	Call of Duty
X-Box	Call of Duty

Welche Person besitzt welche Spiele?

<u>Person</u>	<u>Spiel</u>
Max	Tomb Raider
Max	Call of Duty
Marie	Call of Duty

Aufgabe 03

Aufgabe 03

Bringen Sie das Relationenmodell in die 3. Normalform.

Zusatzaufgabe Normalisierung

Zusatzaufgabe Normalisierung

Für verschiedene Franchises wurden neben Studio auch Filme, Start des Franchise und der Standort des Studios gespeichert. Leider entspricht die Modellierung nicht den Regeln der Normalisierung, daher soll die Relation Schritt für Schritt in die dritte Normalform (3NF) übertragen werden. Bringen Sie dazu zunächst die Relation in die erste Normalform (1NF), danach in die zweite Normalform (2NF) und abschließend in die dritte Normalform (3NF).

Franchise	Standort	Start	Filme
Lucas Film – Star Wars	Kalifornien	1977	Episode IV – Eine neue Hoffnung / Episode V – Das Imperium schlägt zurück / Episode VI – Die Rückkehr der Jedi-Ritter
Warner Bros – Harry Potter	Kalifornien	2001	Stein der Weisen / Die Kammer des Schreckens / Der Gefangene von Askaban
New Line Cinema – Herr der Ringe	Kalifornien	2001	Die Gefährten / Die zwei Türme / Die Rückkehr des Königs

Der Primärschlüssel der Relation ist das Franchise, z.B. Star Wars oder Harry Potter.

Structured Query Language

SQL – Structured Query Language – Hilfe

MySQL Workbench:

<https://www.mysql.com/products/workbench/>

MySQL Workbench Download:

<https://dev.mysql.com/downloads/workbench/>

MySQL Referenzhandbuch: <https://dev.mysql.com/doc/refman/9.4/en/>

SQL-Island: <https://sql-island.informatik.uni-kl.de/>

W3Schools: <https://www.w3schools.com/MySQL/default.asp>

SQL Bolt: <https://sqlbolt.com/>

SQL – Structured Query Language

Datendefinitionssprache (Anlegen / Ändern von Tabellen)

- Data Definition Language (DDL) => CREATE TABLE / ...

Datenmanipulationssprache (Löschen, Anzeigen, Einfügen)

- Data Manipulation Language (DML) => INSERT / SELECT / LOCK / ...

Datenkontrollsprache (Berechtigungen)

- Data Control Language (DCL) => GRANT / COMMIT / ...
-

Anzeigeabfragen werden ggf. auch durch separaten Teilbereich abgedeckt.

Datenabfragesprache (Anzeigen)

- Data Query Language (DQL) => SELECT

SQL – Structured Query Language



Nicht jedes Datenbanksystem unterstützt alle SQL Befehle, die Folien fokussieren sich primär auf MySQL, andere Datenbanksysteme wie z.B. Oracle, MariaDB, PostgreSQL, etc. können sich leicht von der aufgeführten Syntax unterscheiden. Ggf. sind manche Befehle in verschiedenen Systemen gar nicht implementiert und daher nicht möglich.

SQL – Namenskonvention

- GROSSBUCHSTABEN → Reservierte Bezeichner / Befehle
- <x> → Platzhalter für Bezeichner (Buchstaben, Ziffern, Unterstrich) – muss mit Buchstabe beginnen
- [x] → optional
- [x | y | z] → Auswahlliste (entweder oder), optional
- { x | y | z } → Auswahlliste (entweder oder), obligatorisch
- ... → Wiederholung

SQL – Describe

DESCRIBE <Tabelle>;

Field	Type	Null	Key	Default	Extra
CustomerID	int(11)	NO	PRI		auto_increment
CustomerName	varchar(255)	YES			
ContactName	varchar(255)	YES			
Address	varchar(255)	YES			
City	varchar(255)	YES			
PostalCode	varchar(255)	YES			
Country	varchar(255)	YES			

SQL – Datentypen

Zeichenartige Datentypen

- VARCHAR(size)
- CHAR(size)
- TEXT [65.535]
- TINYTEXT [255]
- ENUM(<Wert1>, ...)

Numerische Datentypen

- INT(length) /
INTEGER(length)
- BIGINT(size)
- FLOAT(size, decimals)
- DOUBLE(size, decimals)

Sonstige Datentypen

- DATE [YYYY-MM-DD]
- TIME [HH:MM:SS]
- TIMESTAMP [DATE TIME]
- BOOLEAN [True, False]

SQL – Kommentare

Einzeilige **Kommentare** beginnen mit „--“ (zwei Mal „-“ und Leerzeichen), sie können am Anfang oder am Ende stehen.

-- Kommentar

SELECT * FROM „DUMMY“; -- Kommentar

-- Kommentar SELECT * FROM DUMMY

Mehrzeilige **Kommentare** beginnen mit „/*“ und enden mit „*/“, sie können an jeder beliebigen Stelle stehen und auch nur einzelne Zeilen oder Teile innerhalb einer Zeile kommentieren.

/* Kommentar */

SELECT * FROM „DUMMY“; /* Kommentar */

/* Kommentar SELECT * FROM DUMMY */

SELECT * FROM /* Kommentar */ „DUMMY“;

SQL – CREATE TABLE

SQL – CREATE TABLE

```
CREATE TABLE [ IF NOT EXISTS ] <Tabelle> (  
  <Spalte> <Datentyp> [<Constraint>, ...],  
  ...  
  [ CONSTRAINT <ConstraintName> PRIMARY KEY (<Spalte>, ...) ],  
  [ CONSTRAINT <ConstraintName> CHECK(<Bedingung>, ...) ],  
  [ CONSTRAINT <ConstraintName> FOREIGN KEY (<SpalteAktuelleTabelle>, ...) REFERENCES  
    <Fremdschlüsseltabelle>(<SpalteFremdschlüsseltabelle>, ...) ] )
```

IF NOT EXISTS prüft ob die Tabelle bereits existiert und erstellt sie nur dann, wenn sie noch nicht existiert.
→ Verhindert Fehler bei der Ausführung

```
CREATE TABLE Mitarbeiter (  
  M_ID INTEGER NOT NULL PRIMARY KEY,  
  Vorname VARCHAR(50) NOT NULL,  
  Name VARCHAR(20) NOT NULL,  
  Geburtsdatum DATE,  
  Steuerklasse INTEGER CHECK(Steuerklasse < 7 AND Steuerklasse > 0),  
  Gehalt FLOAT(15,2),  
  Krankenkasse INTEGER,  
  CONSTRAINT FK_Krankenkasse FOREIGN KEY (Krankenkasse) REFERENCES Krankenkassen(Krankenkassen_ID),  
)
```

SQL – CONSTRAINTS

- NOT NULL => Feld ist obligatorisch, muss gefüllt werden
- DEFAULT => Standardwert, wenn kein Wert mitgegeben wird
- AUTO_INCREMENT => Zahlenwert der automatisch hochgezählt wird
=> nur bei INTEGER möglich
- CHECK => Bedingung / Validierungsprüfung
- PRIMARY KEY => Primärschlüssel
- UNIQUE => Keine doppelten Werte in dieser Spalte
- ZEROFILL => führende Leerstellen werden bei Ausgabe mit 0 aufgefüllt
=> UNSIGNED wird implizit ergänzt
=> nur für numerische Datentypen
- UNSIGNED => nur positive Werte möglich
=> nur bei INTEGER möglich

SQL – REFERENCES Constraint

CONSTRAINT <ConstraintName> **FOREIGN KEY** [ConstraintName] (<SpalteAktuelleTabelle>, ...)
REFERENCES <Fremdschlüsseltabelle>(<SpalteFremdschlüsseltabelle>, ...) [**ON DELETE** { **RESTRICT** |
CASCADE | **SET NULL** | **NO ACTION** }] [**ON UPDATE** { **RESTRICT** | **CASCADE** | **SET NULL** | **NO ACTION** }]

Option	Erklärung
RESTRICT	Löschung der abhängigen Einträge wird unterbunden (Standardverhalten)
CASCADE	Löschung aller abhängigen Einträge
SET NULL	Werte in Fremdschlüsseltabelle werden auf NULL gesetzt – nur möglich wenn dort kein NOT NULL definiert wurde
NO ACTION	Identisch zu RESTRICT

SQL – CREATE TABLE Zusatz

CREATE TABLE <Tabelle> **LIKE** <existierende Tabelle>

Mit LIKE wird eine exakte Kopie der Tabelle erzeugt, inklusive Primärschlüssel, Constraints, etc.

CREATE TABLE <Tabelle> **AS SELECT** ...

Mit AS SELECT kann eine Tabelle erzeugt werden, die basierend auf der Query anhand der Selektionsliste definiert wird.

SQL – ALTER / DROP / TRUNCATE TABLE

ALTER TABLE <Tabelle> **DROP COLUMN** | **MODIFY COLUMN** <Spalte> <Datentyp>

ALTER TABLE <Tabelle> **ADD COLUMN** <Spalte> <Datentyp>

ALTER TABLE <Tabelle> **ADD** | **DROP CONSTRAINT** <Constraint> [(<Spalte>, ...)]

TRUNCATE TABLE <Tabelle> → Inhalt einer Datenbanktabelle löschen

DROP TABLE [**IF EXISTS**] <Tabelle> → Datenbanktabelle löschen

IF EXISTS prüft ob die Tabelle existiert und löscht sie nur dann.
→ Verhindert Fehler bei der Ausführung

ALTER TABLE kunden **ADD COLUMN** email char(255);

ALTER TABLE kunden **ADD PRIMARY KEY** (kunden_id);

ALTER TABLE kunden **ADD CONSTRAINT** PK_kunde **PRIMARY KEY** (kunden_id, last_name);

Aufgabe 04

Aufgabe 04

Erstellen Sie die CREATE TABLE Queries für das zuvor definierte Relationenmodell. Achten Sie dabei auf die Zusatzangaben der Attribute aus Aufgabe 01.

SQL – Basic SELECT

SQL – SELECT

SELECT [**DISTINCT** <Spalte>, ...] | <Spalte>, ... | *

FROM <Tabelle>

[**WHERE** <Spalte> = | < | > | <= | >= | <> | **[NOT] BETWEEN** <value> **AND** <value> | **[NOT] IN ()** | **IS [NOT] NULL** | **[NOT] LIKE** <value> [**AND** | **OR** <Spalte> = <value> ...]]

[**ORDER BY** <Spalte> **ASC** | **DESC** [, <Spalte> **ASC** | **DESC**, ...]]

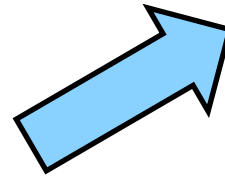
[**LIMIT** <Anzahl Datensätze>]

SELECT mitarbeiter_id, gehalt FROM mitarbeiter **WHERE** gehalt > 2500 **AND** geburtsdatum **BETWEEN** '1999-01-01' **AND** '1999-12-31' **ORDER BY** gehalt **DESC**

SQL – DISTINCT

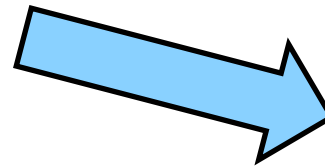
DISTINCT → unterschiedliche Attributwerte

Name	Wohnort
Julia	Mannheim
Katharina	Ludwigshafen
Julia	Mannheim
Klaus	Heidelberg
Katharina	Mannheim



SELECT **DISTINCT** Name, Wohnort;

Name	Wohnort
Julia	Mannheim
Katharina	Ludwigshafen
Klaus	Heidelberg
Katharina	Mannheim



SELECT **DISTINCT** Name;

Name
Julia
Klaus
Katharina

SQL – WHERE Operatoren

Operator	Erklärung	Beispiel
=	gleich	10 = 10
<	Kleiner	10 < 20
>	Größer	10 > 5
<=	Kleiner gleich	10 <= 20 10 <= 10
>=	Größer gleich	10 >= 5 5 >= 5
<>	Ungleich	10 <> 20
[NOT] BETWEEN ... AND ...	Zwischen ... und ... (inklusive der Grenzwerte)	Preis BETWEEN 10 AND 20
[NOT] LIKE	Mustervergleich	Name LIKE ‚A%‘
IS [NOT] NULL	Attributwert ist NULL	Führerschein IS NULL
[NOT] IN (...)	Teil einer vorgegebenen Liste	Primzahl IN (1, 2)

Wird für zeichenartige Literale innerhalb des Literals ein Sonderzeichen benötigt, kann dieses mit dem Escape Zeichen zu einem normalen Zeichen umgewandelt werden. Um in einem zeichenartigen Literal also ein , schreiben zu können muss \ genutzt werden, um einen Backslash schreiben zu können entsprechend \\.

**Escape Zeichen: **

Numerische Literale → 10 (Zahl, ohne weitere Zeichen)

Zeichenartige Literale → ‚ (einfache Anführungszeichen)

**NULL <> NULL → daher
nicht mit „=“ oder „<>“
nach NULL prüfen.**

SQL – LIKE-Wildcards

Wildcard	Erklärung	Beispiel
%	Eine beliebige Anzahl Zeichen	A% → Affe, Angel, Adler / %a% → angeln, zusammen, bauen, Haus / %a → Pizza, rosa
_	Genau ein Zeichen an der gegebenen Position	_o% → Boot, Bodensee / _au_ → Haus, Baum, Kauf / %u__ → Hund, Hugo
[?]	Eines der Zeichen (?) zwischen den Klammern	_[ua]nd → Hund, Rand (findet nicht: Rind)
[!?]	Keines der Zeichen (?) zwischen den Klammern	_[!ua]nd → Rind (findet nicht: Hund, Rand)
[?1-?2]	Alle Zeichen zwischen erstem Zeichen (?1) und zweitem Zeichen (?2)	_[a-i]nd → Rand, Rind (findet nicht: Hund)

SQL – WHERE Verknüpfungsooperatoren

Verknüpfungsooperator	Erklärung
AND	Es müssen beide Bedingungen wahr sein, damit die Gesamtbedingung wahr ist.
OR	Es können beiden Bedingungen oder nur eine der beiden Bedingungen wahr sein, damit die Gesamtbedingung wahr ist.

AND bindet stärker als OR.

Klammern können ebenso genutzt werden um Bedingungen zu kombinieren, Klammern binden stärker als AND und OR.

Preis = 20 AND Alter = 20 OR Alter = 30

- Die Gesamtbedingung ist wahr, wenn das Alter 30 ist oder sowohl Alter 20 als auch Preis 20 ist.
- Indirekte Klammern um AND – (Preis = 20 AND Alter = 20) OR Alter = 30
- Mit entsprechender Klammerung kann z.B. sichergestellt werden, dass der Preis immer 20 sein muss.
 - Preis = 20 AND (Alter = 20 OR Alter = 30)
 - Durch das AND muss Preis immer 20 sein und das Alter kann 20 oder 30 sein.

SQL – WHERE – Anwendungsbeispiel

Fall 1

Alle Studierende welche in Mannheim wohnen und vor dem 01.01.2005 geboren wurden und noch nicht im dritten Semester sind sollen ermittelt werden um mögliche Zuschüsse zu besprechen.

```
SELECT Vorname, Nachname  
FROM Student  
WHERE Wohnort = ,Mannheim' AND Geburtsdatum < ,2005-01-01' AND Semester < 3
```

Fall 2

Die fünf besten Bachelor-Arbeiten der Studierenden sollen entsprechend gewürdigt werden. Es dürfen allerdings nur Studierende teilnehmen, deren Vorname mit „A“ oder „Z“ beginnt oder die in einer Stadt wohnen, deren Name den Begriff „Berg“ enthält.

```
SELECT Vorname, Nachname, Bachelor_Note  
FROM Student  
WHERE Vorname LIKE ,A%' OR Vorname LIKE ,Z%' OR Wohnort LIKE ,%Berg%'  
ORDER BY Bachelor_Note ASC  
LIMIT 5
```

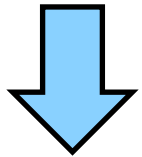
SQL – ORDER BY

ASC (ASCENDING) → Aufsteigende Sortierung

DESC (DESCENDING) → Absteigende Sortierung

SELECT Preis FROM <Tabelle>
ORDER BY Preis **ASC**;

Preis (INT)
20
30
10

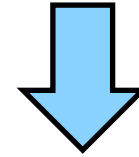


Preis (INT)
10
20
30

Sortierung
abhängig von
Datentyp,
alphanumerische
Datentypen
werden nach
Zeichensatz
sortiert.

SELECT Preis FROM <Tabelle>
ORDER BY Preis **ASC**;

Preis (VARCHAR)
19
2
10

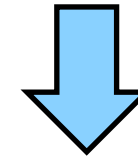


Preis (VARCHAR)
10
19
2

Standardsortierung: ASC

SELECT Name FROM <Tabelle>
ORDER BY Name;

Name (VARCHAR)
Katharina
Julia
Klaus



Name (VARCHAR)
Julia
Katharina
Klaus

SQL – ORDER BY

Mehrere Sortierungen werden in der Reihenfolge priorisiert ausgeführt und bilden Sortierungen innerhalb der höheren Sortierung.

```
SELECT Kurs, Name FROM <Tabelle> ORDER BY Kurs ASC, Name DESC;
```

Kurs	Name
WI SE	Julia
WI SE	Klaus
INF	Julia



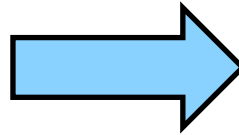
Kurs	Name
INF	Julia
WI SE	Klaus
WI SE	Julia

SQL – LIMIT

LIMIT schränkt die Anzahl der Datensätze ein, die in der Ausgabe resultieren, damit ist es möglich eine exakte Anzahl Datensätze zu selektieren. **LIMIT** ergibt i.d.R. nur in Kombination mit ORDER BY Sinn.

SELECT Name, Note FROM <Tabelle> **ORDER BY** Note **ASC** **LIMIT** 3;

Name	Note
Bernd	1,9
Max	3,4
Marie	1,2
Karl	2,6
Nadine	2,1



Name	Note
Marie	1,2
Bernd	1,9
Nadine	2,1

Aufgabe 05

Aufgabe 05

1. Selektieren Sie alle Attraktionen, die nach 20:00 Uhr schließen und sortieren Sie diese aufsteigend nach der Schließzeit.
2. Sortieren Sie alle Fahrgeschäfte mit einer Höchstgeschwindigkeit von mehr als 50 absteigend nach der Höchstgeschwindigkeit.
3. Welche Shows dauern länger als eine Stunde? Sortieren Sie das Ergebnis aufsteigend nach der Dauer.
4. Welche Besucher sind ab dem 01.01.2000 geboren und haben ein Budget von mindestens 100?
5. Selektieren Sie alle Mitarbeiter mit einem Gehalt zwischen 2500 und 3500 und sortieren Sie das Ergebnis aufsteigend danach.
6. Welche unterschiedlichen Vornamen mit „a“ an zweiter Stelle der Besucher gibt es? Sortieren Sie diese absteigend nach dem Vornamen.
7. Selektieren Sie alle Besucher, deren Vorname mit „S“ beginnt, der Nachname aber kein „S“ enthält und deren Budget mindestens 50 beträgt. Sortieren Sie das Ergebnis absteigend nach dem Budget und beschränken Sie sich auf die höchsten beiden Budgets.
8. Welche Fahrgeschäfte sind für jemanden geeignet, der eine Geschwindigkeit von mindestens 100 erwartet, die Mindestgröße über 100 und unter 190 liegt, G-Kräfte zwischen 3 und 5 herrschen, für Schwangere erlaubt und in der Notiz „Bahn“ oder „Boot“ enthalten sind sowie die Fahrtdauer 04:00 Minuten nicht überschreiten darf. Unabhängig davon ist die Person grundsätzlich an Fahrgeschäften mit dem Mindestalter „0“ interessiert.
9. Welche Mitarbeiter wohnen in einer Straße, die den Begriff „Straße“ beinhaltet und deren Postleitzahl mit „5“ oder „7“ endet?
10. Sortieren Sie alle Attraktionen zunächst aufsteigend nach Öffnungszeit, danach absteigend nach Schließzeit und als letztes aufsteigend anhand des Namens.

SQL – Arithmetische Operatoren

SELECT <Spalte> | <Wert> + | - | / | * | % | DIV <Spalte> | <Wert> FROM <Tabelle>

DIV → Ganzzahldivision → 50 DIV 16 = 3

% → Modulo / Rest → 50 % 16 = 2

SELECT 30 / 15; → 2

SELECT 27 % 14; → 13

SELECT (Preis - 5) * 1.19 AS Inklusive_MWST FROM Produkt;

SQL – Aggregation

SELECT **COUNT**(*) | **COUNT**(**DISTINCT**(<Spalte>)) | **SUM**(<Spalte>) |
 MIN(<Spalte>) | **MAX**(<Spalte>) | **AVG**(<Spalte>) | **GROUP_CONCAT**(<Spalte>)

AS <Spalte>

FROM <Tabelle> [WHERE]

[**GROUP BY** <Spalte>, ...]

[**HAVING** <Aggregation> <Operator> <Vergleichswert> ...]

[ORDER BY]



Aggregatsfunktionen können nicht geschachtelt werden. Daher ist z.B. die Bildung einer Summe über Durchschnittswerte nicht möglich.

→ SUM(AVG (<Spalte>)) → FALSCH!

Diese Problem lässt sich über WITH oder Subselect lösen, dies wird im weiteren Verlauf näher erläutert.

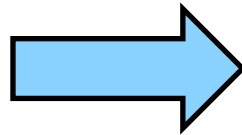
SELECT Matrikelnummer, **SUM**(Fehltage) **AS** Fehltage_Gesamt, **COUNT**(Vorlesung) **AS** Vorlesungen
FROM Semesterplan
GROUP BY Matrikelnummer **HAVING** **SUM**(Fehltage) > 10

SQL – ALIAS

AS ermöglicht es Spalten für die Ausgabe einen neuen Namen zu vergeben. Für berechnete Spalten, z.B. arithmetische Berechnungen oder Aggregatfunktionen ist ein Alias obligatorisch, für Spalten aus der Datenbanktabelle selbst ist ein Alias optional.

SELECT Name **AS** Nachname, Note **AS** Abschlussnote FROM <Tabelle>;

Name	Note
Marie	1,2
Bernd	1,9
Nadine	2,1



Nachname	Abschlussnote
Marie	1,2
Bernd	1,9
Nadine	2,1

SQL – Aggregatsfunktionen

Aggregatsfunktion	Erklärung
COUNT (*)	Anzahl Einträge
COUNT (DISTINCT <Spalte> , ...)	Anzahl unterschiedlicher Einträge
AVG (<Spalte>)	Durchschnittswert (nur für numerische Datentypen)
SUM (<Spalte>)	Summe der Attributwerte (nur für numerische Datentypen)
MIN (<Spalte>)	Minimalwert der Spalte
MAX (<Spalte>)	Maximalwert der Spalte
GROUP_CONCAT (<Spalte>)	Zusammenfügen der Attributwerte durch Komma getrennt → Ausgabe: Julia, Katharina, Klaus

Einfache Arithmetische Operationen können auch innerhalb einer Aggregatsfunktion ausgeführt werden.

NULL Werte werden bei Aggregatsfunktionen ignoriert.

```
SELECT AVG(Note + 0.2) FROM Klausurbewertung;
```

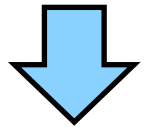
SQL – GROUP BY

GROUP BY ermöglicht es Gruppen von Datensätzen zu bilden, ein entsprechender Indikator ist die Aussage „pro“ (z.B. Anzahl Kunden pro Land). Eine Gruppe wird aus Datensätzen mit gleichen Attributwerten gebildet, dies gilt sowohl für ein Attribut als auch für die Kombination von mehreren Attributen.

```
SELECT Kurs, COUNT(*) AS Anzahl FROM  
Student GROUP BY Kurs;
```

→ Anzahl Studierende pro Kurs

Kurs	Name
WI SE	Julia
WI SE	Klaus
INF	Julia



Kurs	Anzahl
WI SE	2
INF	1

```
SELECT Kurs, Name, AVG(Note) AS Note FROM  
Student GROUP BY Kurs, Name;
```

→ Durchschnittliche Note pro Studierenden und Kurs

Kurs	Name	Note
WI SE	Klaus	2,2
WI SE	Klaus	2,8
INF	Julia	1,8
INF	Julia	3,0
INF	Benjamin	2,5



Kurs	Name	Ø Note
WI SE	Klaus	2,5
INF	Julia	2,4
INF	Benjamin	2,5

**Alle im SELECT
aufgeführten Spalten
müssen Teil der
Gruppierung sein,
sobald zusätzlich
auch Aggregationen
vorhanden sind.**

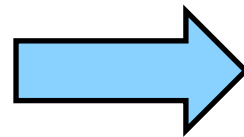
SQL – HAVING

HAVING verhält sich im Aufbau **WHERE**. Im Gegensatz zu **WHERE** werden allerdings nicht die einzelnen Datensätze gefiltert, sondern die Filterung wird erst nach Bildung der Gruppen auf dem Ergebnis dieser Gruppen ausgeführt.

```
SELECT Kurs, COUNT(*) AS Anzahl FROM Student GROUP BY Kurs HAVING COUNT(*) > 1;
```

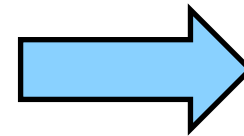
→ Alle Kurse, die mehr als einen Studierenden besitzen.

Kurs	Name
WI SE	Julia
WI SE	Klaus
INF	Julia



Internes Zwischenergebnis
der Gruppierung

Kurs	Anzahl
WI SE	2
INF	1



Ergebnis

Kurs	Anzahl
WI SE	2

HAVING kann auch ohne **GROUP BY** genutzt werden, in diesem Fall bezieht sich die Filterung der Aggregation auf alle Einträge unabhängig einer Gruppe, z.B. ein Durchschnittsgehalt soll nur dann berechnet werden, sollten mindestens 20 Personen angestellt sein.

Aufgabe 06

Aufgabe 06

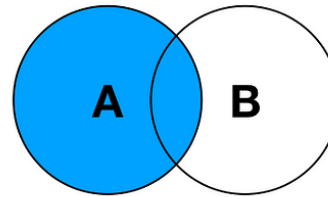
1. Berechnen Sie pro Besucher wie viele Getränke gekauft werden können. Ein Getränk kostet 5,50. Sortieren Sie das Ergebnis anschließend absteigend anhand der meisten Getränke und beschränken Sie sich auf die fünf höchste Anzahl Getränke, die gekauft werden können.
2. Das Gehalt aller Mitarbeiter wird um den Betrag der Hausnummer angehoben. Die Summe aus altem Gehalt und Hausnummer wird anschließend nochmal um weitere 7% erhöht.
3. Wie viele Mitarbeiter arbeiten in dem Freizeitpark?
4. Wie ist das durchschnittliche Gehalt pro Postleitzahl? Sortieren Sie das Ergebnis aufsteigend nach dem durchschnittlichen Gehalt und zeigen Sie nur die niedrigsten drei Durchschnittsgehälter.
5. Welche maximale und minimale Höchstgeschwindigkeit besitzen die Fahrgeschäfte?
6. Was ist die durchschnittliche G-Kraft aller Fahrgeschäfte, die den Begriff „Achterbahn“ in der Notiz enthalten?
7. Wie oft ist eine Kombination aus Vorname und Nachname bei den Besuchern vertreten?
8. Welche Vornamen der Besucher gibt es mindestens zweimal? Sortieren Sie das Ergebnis absteigend nach der Anzahl Vorkommen und beschränken Sie das Ergebnis auf die beliebtesten 10 Namen.
9. In welchen Postleitzahlen liegt das Mindestgehalt bei mindestens 2800?
10. Selektieren Sie alle Arten von Fahrgeschäften welche „Bahn“ oder „Fahrt“ in der Notiz enthalten und zeigen Sie diese in einer einzelnen Zeile an.

SQL – JOIN

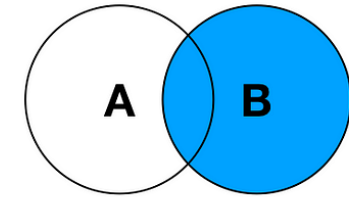
SQL – Join-Arten

Ausschließen der Schnittmenge (Inner Join) wird umgesetzt, indem eine Prüfung auf den Primärschlüssel angegeben wird. Ist der Primärschlüssel in der Ergebnismenge NULL bedeutet dies, dass es keinen entsprechenden Eintrag gibt. Selektiert man also explizit nur Einträge, die in A oder B einen Primärschlüssel mit dem Wert NULL in der Ergebnismenge besitzen (WHERE), wird die Schnittmenge ausgeschlossen.

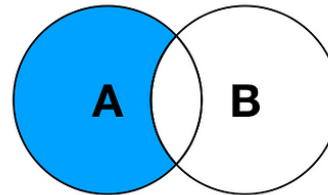
SQL JOINS



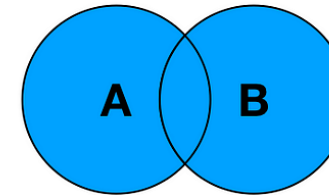
LEFT JOIN



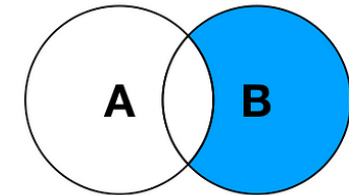
RIGHT JOIN



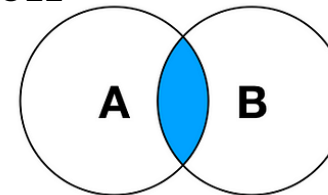
LEFT JOIN EXCLUDING
INNER JOIN
B.Key = NULL



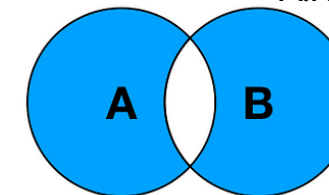
FULL OUTER JOIN



RIGHT JOIN EXCLUDING
INNER JOIN
A.Key = NULL



INNER JOIN



FULL OUTER JOIN EXCLUDING
INNER JOIN
A.Key = NULL OR B.Key = NULL

SQL – Join

SELECT <Alias1|Tabelle1>.<Spalte>, <Alias2|Tabelle2>.<Spalte>

FROM <Tabelle1> [**AS** <Alias1>]

INNER JOIN | **LEFT OUTER JOIN** | **RIGHT OUTER JOIN** |
FULL OUTER JOIN | **CROSS JOIN** | **JOIN** | **NATURAL JOIN** <Tabelle2> [**AS** <Alias2>]

ON <Alias1|Tabelle1>.<Spalte> = <Alias2|Tabelle2>.<Spalte>

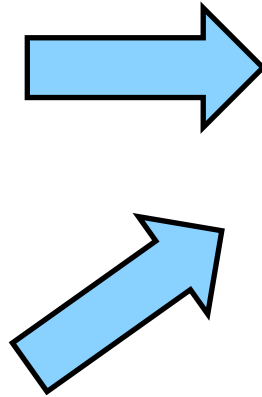
...

SELECT s. Matrikelnummer, s.Name, k.Kurs, k.Kursbezeichnung FROM Student **AS** s
INNER JOIN Kurs **AS** k **ON** s.Matrikelnummer_id = k. Matrikelnummer

SQL – Cross Join (Inner Join)

Film	Studio
Herr der Ringe	New Line Cinema
Star Wars	Lucasfilm
Harry Potter	Warner Bros

Studio	Standort
Lucasfilm	Kalifornien
Warner Bros	Kalifornien
Metro Goldwyn Mayer	Kalifornien



Film	Studio (Film)	Studio (Studio)	Standort
Herr der Ringe	New Line Cinema	Lucasfilm	Kalifornien
Star Wars	Lucasfilm	Lucasfilm	Kalifornien
Harry Potter	Warner Bros	Lucasfilm	Kalifornien
Herr der Ringe	New Line Cinema	Warner Bros	Kalifornien
Star Wars	Lucasfilm	Warner Bros	Kalifornien
Harry Potter	Warner Bros	Warner Bros	Kalifornien
Herr der Ringe	New Line Cinema	Metro Goldwyn Mayer	Kalifornien
Star Wars	Lucasfilm	Metro Goldwyn Mayer	Kalifornien
Harry Potter	Warner Bros	Metro Goldwyn Mayer	Kalifornien

Kartesisches Produkt – Kombination aller
Tupel von A mit allen Tupeln von B.

```
SELECT f.Film, s.Studio, s.Standort
```

```
FROM Film AS f
```

```
CROSS JOIN | JOIN Studio AS s
```

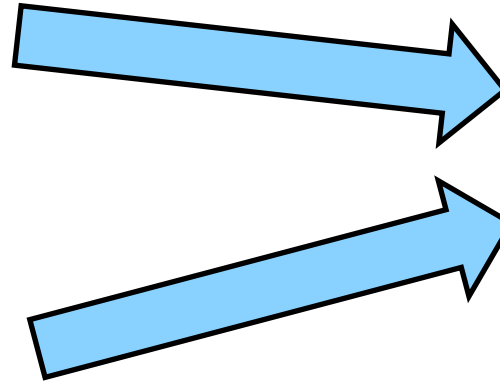


JOIN agiert in MySQL unterschiedlich, abhängig ob eine ON-Bedingung angegeben wird oder nicht – ohne ON → CROSS JOIN. Daher wird empfohlen, die ausgeschriebene Version des jeweiligen Joins zu nutzen.

SQL – Theta Join / Equijoin (Inner Join)

Film	Studio
Herr der Ringe	New Line Cinema
Star Wars	Lucasfilm
Harry Potter	Warner Bros

Studio	Standort
Lucasfilm	Kalifornien
Warner Bros	Kalifornien
Metro Goldwyn Mayer	Kalifornien



Film	Studio	Standort
Star Wars	Lucasfilm	Kalifornien
Harry Potter	Warner Bros	Kalifornien

Kartesisches Produkt mit Selektion (Filter).
Bei Angabe der Operatoren (ON-Bedingung)
auch Equijoin genannt. Ermittelt
Schnittmenge anhand der Bedingung.

SELECT f.Film, s.Studio, s.Standort

FROM Film **AS** f

INNER JOIN | **JOIN** Studio **AS** s

ON f.Studio = s.Studio

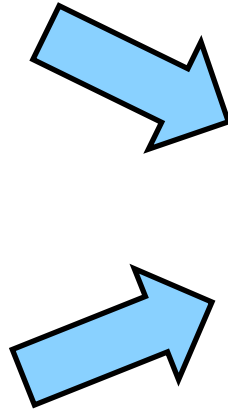


JOIN agiert in MySQL unterschiedlich, abhängig ob eine ON-Bedingung angegeben wird oder nicht – mit ON → INNER JOIN. Daher wird empfohlen, die ausgeschriebene Version des jeweiligen Joins zu nutzen.

SQL – Theta Join / Non-Equijoin (Inner Join)

Film	Studio
Herr der Ringe	New Line Cinema
Star Wars	Lucasfilm
Harry Potter	Warner Bros

Studio	Standort
Lucasfilm	Kalifornien
Warner Bros	Kalifornien
Metro Goldwyn Mayer	Kalifornien



Film	Studio (Film)	Studio (Studio)	Standort
Herr der Ringe	New Line Cinema	Lucasfilm	Kalifornien
Harry Potter	Warner Bros	Lucasfilm	Kalifornien
Herr der Ringe	New Line Cinema	Warner Bros	Kalifornien
Star Wars	Lucasfilm	Warner Bros	Kalifornien
Herr der Ringe	New Line Cinema	Metro Goldwyn Mayer	Kalifornien
Star Wars	Lucasfilm	Metro Goldwyn Mayer	Kalifornien
Harry Potter	Warner Bros	Metro Goldwyn Mayer	Kalifornien

SELECT f.Film, s.Studio, s.Standort

FROM Film **AS** f

INNER JOIN | **JOIN** Studio **AS** s

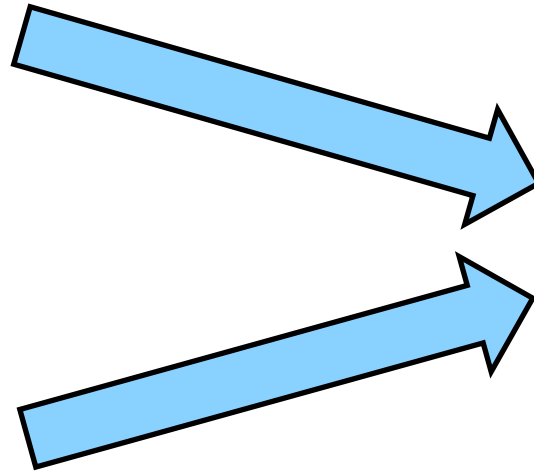
ON f.Studio <> s.Studio

Im Gegensatz zum Equijoin wird hier nicht auf Gleichheit geprüft, sondern eine andere Bedingung, z.B. kleiner oder ungleich.

SQL – Natural Join

Film	Studio
Herr der Ringe	New Line Cinema
Star Wars	Lucasfilm
Harry Potter	Warner Bros

Studio	Standort
Lucasfilm	Kalifornien
Warner Bros	Kalifornien
Metro Goldwyn Mayer	Kalifornien



Film	Studio	Standort
Star Wars	Lucasfilm	Kalifornien
Harry Potter	Warner Bros	Kalifornien

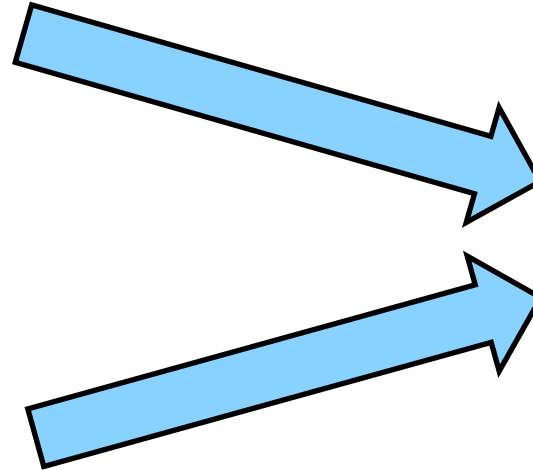
Sonderform des Equijoin, ON-Bedingung wird über Attribute mit gleichen Namen durchgeführt.

```
SELECT f.Film, s.Studio, s.Standort  
FROM Film AS f  
NATURAL JOIN Studio AS s
```

SQL – Left Outer Join

Film	Studio
Herr der Ringe	New Line Cinema
Star Wars	Lucasfilm
Harry Potter	Warner Bros

Studio	Standort
Lucasfilm	Kalifornien
Warner Bros	Kalifornien
Metro Goldwyn Mayer	Kalifornien



Film	Studio	Standort
Star Wars	Lucasfilm	Kalifornien
Harry Potter	Warner Bros	Kalifornien
Herr der Ringe		

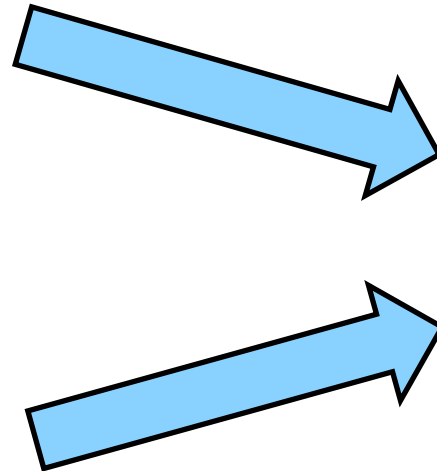
Alle Einträge der linken Tabelle werden übernommen, ungeachtet ob in der rechten Tabelle etwas gefunden wird, sollte etwas gefunden werden, werden die Tupel verbunden.

```
SELECT f.Film, s.Studio, s.Standort  
FROM Film AS f  
LEFT OUTER JOIN Studio AS s  
ON f.Studio = s.Studio
```

SQL – Right Outer Join

Film	Studio
Herr der Ringe	New Line Cinema
Star Wars	Lucasfilm
Harry Potter	Warner Bros

Studio	Standort
Lucasfilm	Kalifornien
Warner Bros	Kalifornien
Metro Goldwyn Mayer	Kalifornien



Film	Studio	Standort
Star Wars	Lucasfilm	Kalifornien
Harry Potter	Warner Bros	Kalifornien
	Metro Goldwyn Mayer	Kalifornien

Alle Einträge der rechten Tabelle werden übernommen, ungeachtet ob in der linken Tabelle etwas gefunden wird, sollte etwas gefunden werden, werden die Tupel verbunden.

```
SELECT f.Film, s.Studio, s.Standort
```

```
FROM Film AS f
```

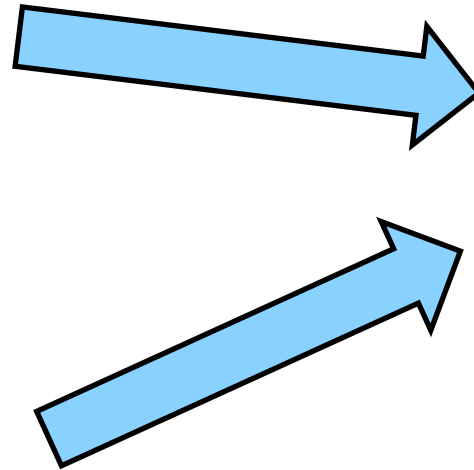
```
RIGHT OUTER JOIN Studio AS s
```

```
ON f.Studio = s.Studio
```

SQL – Full Outer Join

Film	Studio
Herr der Ringe	New Line Cinema
Star Wars	Lucasfilm
Harry Potter	Warner Bros

Studio	Standort
Lucasfilm	Kalifornien
Warner Bros	Kalifornien
Metro Goldwyn Mayer	Kalifornien



Film	Studio	Standort
Star Wars	Lucasfilm	Kalifornien
Harry Potter	Warner Bros	Kalifornien
Herr der Ringe	New Line Cinema	
	Metro Goldwyn Mayer	Kalifornien

Alle Einträge der linken und der rechten Tabelle werden übernommen, ungeachtet ob in der jeweils anderen Tabelle etwas gefunden wird, sollte etwas gefunden werden, werden die Tupel verbunden.

```
SELECT f.Film, s.Studio, s.Standort
```

```
FROM Film AS f
```

```
FULL OUTER JOIN Studio AS s
```

```
ON f.Studio = s.Studio
```



MySQL unterstützt
Full Outer Join nicht.

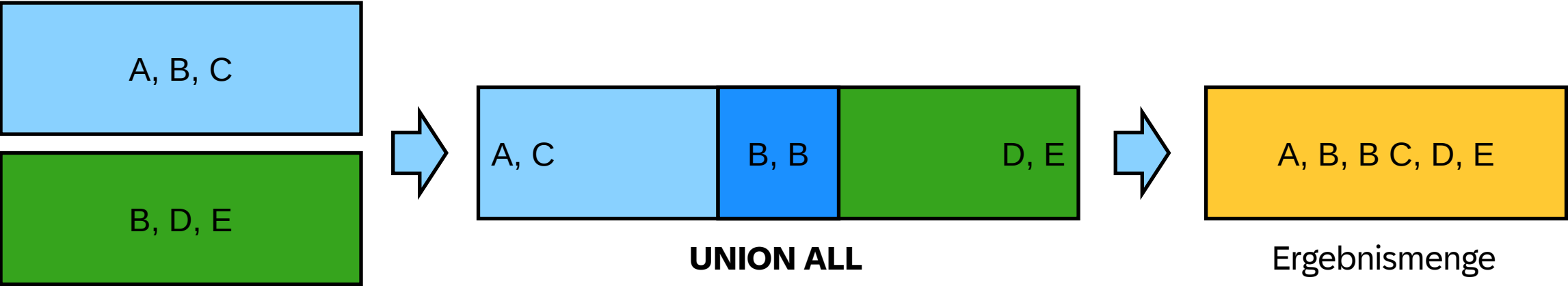
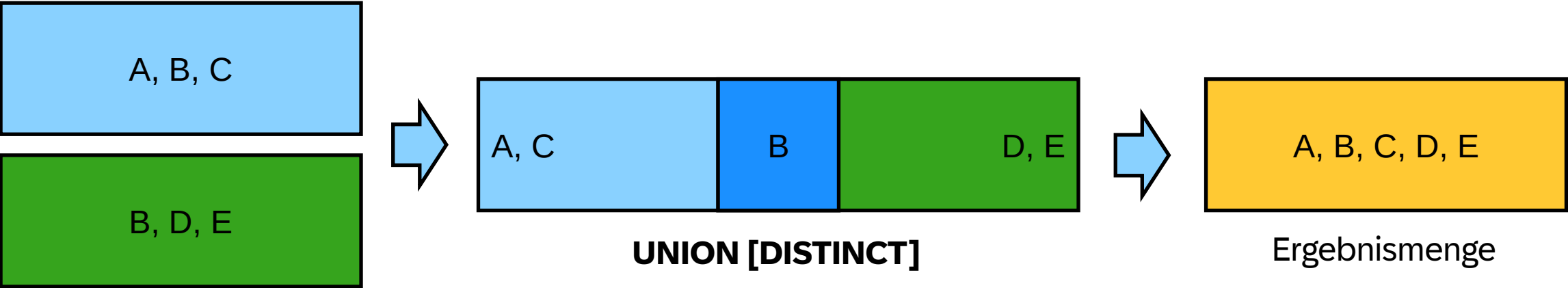
Aufgabe 07

Aufgabe 07

1. Zeigen Sie neben der Postleitzahl auch den Namen der Stadt bei der Adresse für alle Mitarbeiter an, welche vor dem 01.01.2000 geboren sind und eine ungerade Hausnummer besitzen.
2. Zeigen Sie für alle Besucher den Eintritt und berechnen Sie anhand des übrigen verfügbaren Budgets und des Eintritts das insgesamt vorhandene Budget, sofern der Vorname länger als 5 Zeichen ist. Sortieren Sie das Ergebnis absteigend anhand des Gesamtbudgets und schränken Sie das Ergebnis auf die 20 höchsten Budgets ein.
3. Zeigen Sie zu allen Shows auch den Namen der Attraktion an sowie die Beschreibung der Art.
4. Berechnen Sie den durchschnittlich bezahlten Eintrittspreis pro Person.
5. Gibt es Show-Arten zu denen derzeit keine Show verfügbar ist? Falls ja, zeigen Sie diese.
6. Zeigen Sie alle Fahrgeschäft-Arten, unabhängig davon, ob es Fahrgeschäfte der Art gibt und die berechnen Sie die Anzahl Fahrgeschäfte pro Art. Sortieren Sie das Ergebnis absteigend nach der Anzahl Fahrgeschäfte.
7. Welche Besucher mit welchen Vornamen und Nachnamen haben welche Attraktionen, welche durch den Namen beschrieben werden, besucht.
8. Gibt es Besucher, die ein Fahrgeschäft mehrfach besucht haben?
9. Wie viele Besucher haben eine Attraktion besucht, zeigen Sie dazu den Namen der Attraktion und nur Attraktionen, welche mindestens 3 Besucher hatten.
10. Wie viel Geld kostet das Betreiben pro Attraktion. Zeigen Sie dazu den Namen der Attraktion.
11. Wie hoch ist das durchschnittliche Gehalt pro Fahrgeschäft-Art. Zeigen Sie dazu den Namen der Fahrgeschäft-Art ebenfalls.
12. Zeigen Sie alle Möglichkeiten, welcher Besucher welche Attraktion besuchen könnte.

SQL – UNION

SQL – Union-Arten



SQL – UNION

```
SELECT [<Wert> AS <Name>, ...] | <Spalte>, ... FROM <Tabelle1> [ WHERE ] [ GROUP BY ] [ HAVING ]  
UNION [ DISTINCT ] | UNION ALL  
SELECT [<Wert> AS <Name>, ...] | <Spalte>, ... FROM <Tabelle2> [ WHERE ] [ GROUP BY ] [ HAVING ]  
...  
[ ORDER BY ]
```

- **Gleiche Anzahl Spalten**
- **Gleicher Datentyp**
- **Gleiche Reihenfolge / Gleicher Name**

```
SELECT name FROM kunden UNION SELECT name FROM mitarbeiter ORDER BY name;
```

```
SELECT name, vorname FROM kunden WHERE land = 'DE' UNION ALL SELECT name,  
vorname FROM mitarbeiter WHERE land = 'DE' ORDER BY City;
```

SQL – UNION

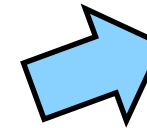
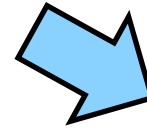
```
SELECT f.Film, f.Studio  
FROM Film AS f
```

UNION

```
SELECT s.Film, s.Studio  
FROM Serie AS s
```

Film	Studio
Herr der Ringe	New Line Cinema
Star Wars	Lucasfilm
Harry Potter	Warner Bros

Serie	Studio
The Rookie	ABC Signature
The Clone Wars	Lucasfilm
Harry Potter	Warner Bros



Alle Tupel der beiden Tabellen werden in eine Tabelle zusammengeführt, Duplikate werden entfernt.

Unterhaltung	Studio
Herr der Ringe	New Line Cinema
Star Wars	Lucasfilm
Harry Potter	Warner Bros
The Rookie	ABC Signature
The Clone Wars	Lucasfilm

SQL – UNION ALL

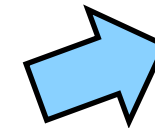
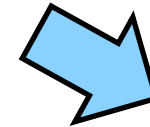


```
SELECT f.Film, f.Studio  
FROM Film AS f  
UNION ALL
```

```
SELECT s.Film, s.Studio  
FROM Serie AS s
```

Film	Studio
Herr der Ringe	New Line Cinema
Star Wars	Lucasfilm
Harry Potter	Warner Bros

Serie	Studio
The Rookie	ABC Signature
The Clone Wars	Lucasfilm
Harry Potter	Warner Bros



Alle Tupel der beiden Tabellen werden in eine Tabelle zusammengeführt, Duplikate werden beibehalten.

Unterhaltung	Studio
Herr der Ringe	New Line Cinema
Star Wars	Lucasfilm
Harry Potter	Warner Bros
The Rookie	ABC Signature
The Clone Wars	Lucasfilm
Harry Potter	Warner Bros

SQL – Union als Ersatz für Full Outer Join

Film	Studio
Herr der Ringe	New Line Cinema
Star Wars	Lucasfilm
Harry Potter	Warner Bros

Studio	Standort
Lucasfilm	Kalifornien
Warner Bros	Kalifornien
Metro Goldwyn Mayer	Kalifornien

RIGHT OUTER

LEFT OUTER

Film	Studio	Standort
Star Wars	Lucasfilm	Kalifornien
Harry Potter	Warner Bros	Kalifornien
	Metro Goldwyn Mayer	Kalifornien

Film	Studio	Standort
Star Wars	Lucasfilm	Kalifornien
Harry Potter	Warner Bros	Kalifornien
Herr der Ringe		

```
SELECT f.Film, s.Studio, s.Standort
FROM Film AS f
LEFT OUTER JOIN Studio AS s
ON f.Studio = s.Studio
UNION
```

```
SELECT f.Film, s.Studio, s.Standort
FROM Film AS f
RIGHT OUTER JOIN Studio AS s
ON f.Studio = s.Studio
```

UNION

Die doppelten Einträge werden jeweils nur einmal übernommen.

Film	Studio	Standort
Star Wars	Lucasfilm	Kalifornien
Harry Potter	Warner Bros	Kalifornien
Herr der Ringe	New Line Cinema	
	Metro Goldwyn Mayer	Kalifornien

SQL – EXCEPT / INTERSECT



SQL – EXCEPT

SELECT [<Wert> AS <Name>, ...] | <Spalte>, ... FROM <Tabelle1> [WHERE] [GROUP BY] [HAVING]

EXCEPT [DISTINCT] | EXCEPT ALL

SELECT [<Wert> AS <Name>, ...] | <Spalte>, ... FROM <Tabelle2> [WHERE] [GROUP BY] [HAVING]
[ORDER BY]

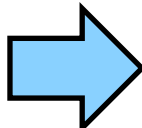
SELECT f.Film **AS**
Unterhaltung, f.Studio
FROM Film **AS** f

EXCEPT

SELECT s.Film, s.Studio
FROM Serie **AS** s

Film	Studio
Herr der Ringe	New Line Cinema
Star Wars	Lucasfilm
Harry Potter	Warner Bros

Serie	Studio
The Rookie	ABC Signature
The Clone Wars	Lucasfilm
Harry Potter	Warner Bros



Differenz

→ Alle Tupel aus Film, sofern sie nicht in Serie enthalten sind.

Unterhaltung	Studio
Herr der Ringe	New Line Cinema
Star Wars	Lucasfilm



SQL – INTERSECT

SELECT [<Wert> AS <Name>, ...] | <Spalte>, ... FROM <Tabelle1> [WHERE] [GROUP BY] [HAVING]

INTERSECT [DISTINCT] | INTERSECT ALL

SELECT [<Wert> AS <Name>, ...] | <Spalte>, ... FROM <Tabelle2> [WHERE] [GROUP BY] [HAVING]
[ORDER BY]

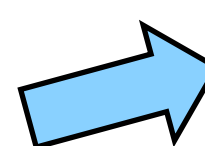
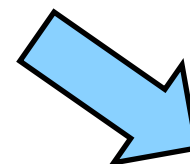
SELECT f.Film **AS**
Unterhaltung, f.Studio
FROM Film **AS** f

INTERSECT

SELECT s.Film, s.Studio
FROM Serie **AS** s

Film	Studio
Herr der Ringe	New Line Cinema
Star Wars	Lucasfilm
Harry Potter	Warner Bros

Serie	Studio
The Rookie	ABC Signature
The Clone Wars	Lucasfilm
Harry Potter	Warner Bros



Schnitt

→ Alle Tupel aus Film, sofern sie on Serie enthalten sind.

Unterhaltung	Studio
Harry Potter	Warner Bros

Aufgabe 08

Aufgabe 08

- ✓ 1. Welche unterschiedlichen Vornamen sind in der Gesamtmenge der Vornamen aus Mitarbeitern und Besuchern vorhanden? Sortieren Sie die Vornamen absteigend.
2. Welche Arten von sowohl Shows als auch Fahrgeschäften bietet der Freizeitpark, dabei sollen nur Fahrgeschäfte betrachtet werden, welche in der Beschreibung den Begriff „Fahrt“ nicht enthalten.
3. Fahrgeschäfte und Shows sollen in einem Ergebnis zusammengefasst werden, dabei soll unterschieden werden, ob es sich um eine Show oder ein Fahrgeschäft handelt. Ebenso sollen alle Informationen angegeben werden sowie nur Fahrgeschäfte betrachtet werden, die ein Mindestalter von maximal 10 Jahren besitzen. Sortieren Sie das Ergebnis aufsteigend nach der Dauer und beschränken Sie das Ergebnis auf die ersten 15 Attraktionen (Hinweis: Die Relation „Attraktion“ darf nicht genutzt werden).
4. Selektieren Sie alle Fahrgeschäfte und Arten von Fahrgeschäften, unabhängig davon ob zu einem Fahrgeschäft eine Art vorhanden ist oder eine Art auch ein Fahrgeschäft besitzt.
- ✓ 5. Welche Vornamen sind bei Besucher vertreten, aber nicht bei Mitarbeitern?
- ✓ 6. Welche Vornamen sind bei Besuchern und Mitarbeitern vertreten?

SQL – Funktionen

SQL – Funktionen – Zeichenketten

Funktionsname	Erklärung
UPPER (<Wert>)	Umwandlung der Zeichenkette in Uppercase
LOWER (<Wert>)	Umwandlung der Zeichenkette in Lowercase
CONCAT (<Wert>, ...)	Verbindung von Zeichenketten
CHAR_LENGTH (<Wert>)	Zeichenanzahl der Zeichenkette
REPLACE (<Zeichenkette>, <alt>, <neu>)	Ersetzt alt durch neu in der angegebenen Zeichenkette
SUBSTRING (<Zeichenkette>, <Startindex>, <Zeichenanzahl>)	Extrahiert Teil der Zeichenkette ab Startindex mit der Länge der Zeichenanzahl (inklusive Startindex)
CONCAT_WS (<Separator>, <Wert>, ...)	Verbindung von Zeichenketten mit einem Separator zwischen den Werten
TRIM (<Wert>)	Leerzeichen an Beginn und Ende werden entfernt
RTRIM (<Wert>)	Leerzeichen am Ende werden entfernt
...	...

SELECT **SUBSTRING**("DHBW Mannheim", 8, 3) AS ExtractString; → nnh

SELECT **REPLACE**("DHBW Mannheim", "DHBW", "Duale Hochschule"); → Duale Hochschule Mannheim

SQL – Funktionen – Zahlen

Funktionsname	Erklärung
ABS (<Wert>)	Betragsfunktion
CEIL (<Wert>)/ CEILING (<Wert>)	Immer Aufrunden
FLOOR (<Wert>)	Immer Abrunden
SQRT (<Wert>)	Quadratwurzel
ROUND (<Wert>, <Anzahl Nachkommastellen>)	Runden des Wertes auf Anzahl Nachkommastellen
POW (<Basis>, <Exponent>)	Potenzfunktion
RAND ()	Zufallszahl (Gleitkommazahl) zwischen 0 und 1
...	...

SELECT **ABS**(-25); → 25

SELECT **CEIL**(25.4); → 26

SELECT **FLOOR**(1 + **RAND**() * 100); → Zahl zwischen 1 und 100

SQL – Sonstige Funktionen

Funktionsname	Erklärung
DATEDIFF (<Datum1>, <Datum2>)	Differenz in Tagen zwischen Datum 1 (Beginn) und Datum 2 (Ende)
DAYNAME (<Datum>)	Name des Wochentags des Datums
IFNULL (<Attributname>, <Ersatzwert>)	Wenn der Attributwert NULL ist, ist die Rückgabe der Ersatzwert, andernfalls wird der eigentliche Attributwert zurückgegeben
IF (<Bedingung>, <wahr>, <falsch>)	Prüft die Bedingung und liefert den entsprechenden Wert zurück
CURRENT_DATE () / CURDATE ()	Aktuelles Datum
CAST (<Wert> AS <Datentyp>)	Konvertierung eines Wertes in einen anderen Datentypen
TIMESTAMPDIFF (<Einheit>, <Zeitstempel1>, <Zeitstempel2>)	Differenz abhängig der Einheit zwischen Zeitstempel 1 (Beginn) und Zeitstempel 2 (Ende)
...	...

```
SELECT IF(500 > 400, „Yes“, „No“); → Yes
```

```
SELECT CURRENT_DATE();
```

```
SELECT CAST(,2000-01-01' AS DATE )
```

SQL – TIMESTAMPDIFF

Einheit	Erklärung
SECOND	Differenz in vollen Sekunden
MINUTE	Differenz in vollen Minuten
hour	Differenz in vollen Stunden
DAY	Differenz in vollen Tagen
WEEK	Differenz in vollen Wochen
MONTH	Differenz in vollen Monaten
QUARTER	Differenz in vollen Quartalen (3 Monate)
YEAR	Differenz in vollen Jahren

SELECT TIMESTAMPDIFF(hour, '2025-04-01 08:00:00', '2025-04-01 10:30:00'); → 2

SELECT TIMESTAMPDIFF(YEAR, '2010-01-01', '2025-01-01'); → 15

SELECT TIMESTAMPDIFF(DAY, '2025-01-01', '2025-04-01'); → 90

SQL – Funktionen – Anwendungsbeispiel

Fall 1

Für alle Studierenden sollen Vorname und Nachname zusammengefasst und ermittelt werden, wie viele Tage sie bereits an der DHBW eingeschrieben sind.

```
SELECT CONCAT_WS( , , Vorname, Nachname ) AS Name, DATEDIFF( CURRENT_DATE( ), Eintrittsdatum ) AS Anzahl_Tage  
FROM Student
```

Fall 2

Aufgrund technischer Fehler bei der Eintragung der Noten, sollen diese alle auf die nächste ganze Zahl abgerundet werden.

```
SELECT FLOOR( Note ) AS Neue_Note  
FROM Klausurbewertung
```

Fall 3

Der Kurs „Wirtschaftsinformatik Software Engineering“ soll umbenannt werden in „BWL-Informatik Software Engineering“.

```
SELECT REPLACE( Kursname, ,Wirtschaftsinformatik‘, ,BWL-Informatik‘ ) AS Kursname  
FROM Kurs
```

SQL – CASE

```
SELECT      CASE
            WHEN <Bedingung> THEN <Ausgabe>
            [...]
            ELSE <Ausgabe>
            END AS <Spaltenname>
FROM <Tabelle>
```

SQL – CASE – Anwendungsbeispiel

Für verschiedene Noten soll eine entsprechende Beschreibung angegeben werden. Die Noten sollen daher in die klassischen Noten „sehr gut“ bis „ungenügend“ umgewandelt werden.

```
SELECT Note,  
CASE  
    WHEN Note < 1.45 THEN "sehr gut"  
    WHEN Note < 2.45 THEN "gut"  
    WHEN Note < 3.45 THEN "befriedigend"  
    WHEN Note <= 4 THEN "ausreichend"  
    ELSE "ungenügend"  
END AS Notentext  
FROM Klausurbewertung;
```

Da Noten schlechter als 4 nicht möglich sind und automatisch als nicht bestanden gelten, wird nicht weiter zwischen „mangelhaft“ und „ungenügend“ unterschieden.

Aufgabe 09

Aufgabe 09

- ✓ 1. Die Preise sollen um 10% angehoben und auf den nächsthöheren vollen Betrag gerundet werden.
- ✓ 2. Das Gehalt soll um 3% erhöht werden und auf den nächstniedrigeren Betrag gerundet werden.
- ✓ 3. An welchem Wochentag sind Besucher geboren?
- ✓ 4. Besucher, deren Budget über 100 liegt sollen als „reich“ angezeigt werden.
- ✓ 5. Um Städtenamen mit Abkürzungen zu versehen, sollen die ersten 3 Zeichen der Namen angezeigt werden.
6. Die Beschreibungstexte der Eintrittsbeträge enthalten zu Beginn und am Ende leider Leerzeichen, damit diese richtig gedruckt werden können, müssen diese Leerzeichen entfernt werden.
7. Abhängig der G-Kräfte der Fahrgeschäfte soll ermittelt werden, wie hoch die Belastung ist. Ab 4 ist die Belastung „sehr hoch“, ab 3 „hoch“, ab 2 „angemessen“ und unter 2 „niedrig“.
8. Welches Alter haben die Mitarbeiter? Abhängig des Alters soll zusätzlich die entsprechende Generation ausgegeben werden.
9. Für neue Visitenkarten der Mitarbeiter sollen Vorname und Nachname zusammengefasst werden, sowie alle Informationen der Adresse durch Komma getrennt ebenfalls zusammengefasst werden.
10. Für Attraktionen müssen neue Schilder gedruckt werden, wie lange sind die jeweiligen Namen und Notizen? Ersetzen Sie vor der Berechnung der Anzahl der Zeichen den Begriff „Achterbahn“ in den Notizen durch „Coaster“.

SQL – INSERT

SQL – INSERT

INSERT INTO <Tabelle>

VALUES (<Value>, <Value>, ...) [, ...]

→ Reihenfolge muss mit DB übereinstimmen

INSERT INTO <Tabelle> (<Spalte>, <Spalte>, ...)

VALUES (<Value>, <Value>, ...) [, ...]

→ Spalten werden anhand der beiden Klammern passend zugeordnet

INSERT INTO Person **VALUES**(92, ,SAP', ,Andreas Nicola', null, ,Walldorf', ,69190', ,DE');

SQL – INSERT & SELECT

```
INSERT INTO <Tabelle_Neu>  
SELECT * FROM <Tabelle_Alt>  
WHERE <Bedingung>
```

→ Selektiert alle Spalten und fügt sie in die neue Tabelle, Tabellen müssen gleich sein

```
INSERT INTO <Tabelle_Neu> (<Spalte>, <Spalte>, ...)  
SELECT <Spalte>, <Spalte>, ... FROM <Tabelle_Alt>  
WHERE <Bedingung>;
```

→ Selektiert die aufgeführten Spalten und fügt sie den bei INSERT INTO gewählten Spalten hinzu

```
INSERT INTO Person_Neu SELECT * FROM Person WHERE land = ,DE‘;
```

SQL – UPDATE

SQL – UPDATE

UPDATE <Tabelle>

SET <Spalte> = <Value>, ...

WHERE <Spalte> = <Value>

Die WHERE Bedingung
kann auch eine reguläre
Bedingung mit OR, AND,
etc. sein.



Ohne WHERE Bedingung werden
alle Datensätze angepasst!

UPDATE Person **SET** Adresse = ‚Dietmar Hopp Allee 16‘ **WHERE** Personenummer = 92;

SQL – DELETE

SQL – DELETE

DELETE FROM <Tabelle>

WHERE <Spalte> = <Value>

Die WHERE Bedingung kann auch eine reguläre Bedingung mit OR, AND, etc. sein.



Ohne WHERE Bedingung wird die komplette Tabelle geleert (nicht gelöscht)!

DELETE FROM Personen **WHERE** Personennummer = 92;

DELETE FROM Personen;

Aufgabe 10

Aufgabe 10

Als Vorbereitung für die Aufgaben führen Sie das Skript „**Datenbanken_I_Datenmodell_AmusementPark_Student_Setup.sql**“ aus. Dieses erstellt die Datenbanktabellen und fügt Daten hinzu.

- ✓ 1. Fügen Sie zwei Fahrgeschäfte und eine Show hinzu.
2. Ergänzen Sie sich als Mitarbeiter/-in, Sie erhalten nur das Mindestgehalt. Sie betreuen ein Fahrgeschäft.
3. Ergänzen Sie sich als Besucher, Sie erhalten ein Budget von 30. Besuchen Sie drei Attraktionen.
- ✓ 4. Durch eine Hochzeit heißt „Anna Schmidt“ nun „Anna Schmidt Schmitt“ und „Patrick Schmitt“ heißt „Patrick Schmitt Schmidt“, beides sind Besucher des Freizeitparks.
5. Die Gehälter werden alle angepasst und um 15% erhöht, sofern der Mitarbeiter jünger als 50 Jahre ist. Andernfalls wird das Gehalt nur um 8% erhöht.
6. Alle Fahrgeschäfte mit „Bahn“ in den Notizen erhalten ein Mindestalter von 10.
7. „Marc Dietrich“ hat leider gekündigt und muss daher aus der Datenbanktabelle gelöscht werden.
8. Der Freizeitpark darf auf Grund von Gesetzesänderungen leider keine Kampfkunst-Shows mehr aufführen.

Um ggf. vorhandene Primärschlüssel Verletzungen oder Fremdschlüsselprüfungen für die Aufgaben zu umgehen, diese beiden Befehle ausführen:

SET SQL_SAFE_UPDATES=0;
SET FOREIGN_KEY_CHECKS=0;



Immer nur nach Absprache ausführen, kann Inkonsistenzen in den Daten hervorbringen.



SQL – View

SQL – View

Ein SQL-View ist ein Template für eine SQL Query und stellt eine virtuelle Tabelle für eine SQL Query dar. Eine View enthält daher selbst keine Daten, da er immer nur bei tatsächlicher Verwendung / bei Aufruf erzeugt wird. Daher enthält das Ergebnis auch immer die aktuellsten Daten.

Eine View wird i.d.R. dafür genutzt häufig verwendete SQL Queries zu speichern und dann einen SELECT auf der View abzusetzen statt auf die Datenbanktabellen selbst.

```
CREATE VIEW [ IF NOT EXISTS ] <View_Name> AS  
<SQL Select Query>  
;
```

Löschen einer View:

```
DROP VIEW [ IF EXISTS ] <View_Name>;
```

Die SQL SELECT Query kann Joins, Unions, Aggregationen, berechnete Felder, WHERE, GROUP BY, HAVING, etc. enthalten. Der Befehl CREATE VIEW erzeugt dann anhand der Query ein Template von dem selektiert werden kann.

```
SELECT * FROM <View_Name>;
```

IF EXISTS | IF NOT EXISTS können auch bei Views genutzt werden.

SQL – View – Anwendungsbeispiel

Es gibt die Datenbanktabellen „Studierende“, „Belegung“ und „Kurse“. Möchte man nun wissen welcher Studierende welchen Kurs belegt hat (Klarnamen und Beschreibungen der Kurse, IDs sind in „Belegung“ bereits vorhanden) muss man alle drei Tabellen durch JOIN verknüpfen. Benötigt man diese Informationen mehrfach muss man diese Query mehrfach ausführen. Einfacher wäre es daher, ein Template zu erzeugen welches genau diese Query beinhaltet und von diesem Template Daten zu lesen.

Schritt 1 – Erstellen der View, welche die drei Tabellen per JOIN kombiniert

```
CREATE VIEW Kursbelegung AS  
SELECT S.Vorname, S.Nachname, K.Kursbeschreibung, K.Kursnummer,  
B.Matrikelnummer  
FROM Studierende AS S  
INNER JOIN Belegung AS B ON B.Matrikelnummer = S.Matrikelnummer  
INNER JOIN Kurse AS K ON B.Kursnummer = K.Kursnummer;
```

Schritt 2 – Selektieren der Daten aus der View

```
SELECT * FROM Kursbelegung;
```

Aufgabe 11

Aufgabe 11

Als Vorbereitung für die Aufgaben führen Sie das Skript „[Datenbanken_I_Datenmodell_AmusementPark_Student_Setup.sql](#)“ aus. Dieses erstellt die Datenbanktabellen und fügt Daten hinzu.

1. Erstellen Sie eine View, welche alle Besucher und deren bezahlten Eintrittspreis enthält.
2. Erstellen Sie eine View, welche alle Mitarbeiter und deren Wohnort enthält.
3. Erstellen Sie eine View, welche alle Informationen zu allen Attraktionen (Fahrgeschäfte und Shows inklusive der jeweiligen Artenbeschreibungen) enthält.
4. Erstellen Sie eine View, welche alle Attraktionen enthält, die von mindestens zwei Mitarbeitern betreut werden. Zeigen Sie auch an um welchen Typ von Attraktion es sich dabei handelt.
5. Erstellen Sie eine View, welche alle Besuche enthält, inklusive der Besuchernamen und beschränken Sie das Ergebnis auf die Besucher, die mindestens ein Budget von 100 besitzen.

SQL – Subquery

SQL – Subquery

Subquery als **SELECT** in **WHERE**-Bedingung:

SELECT <Spalte> **FROM** <Tabelle1>

WHERE <Spalte> <Operator>

(**SELECT** <Spalte> **FROM** <Tabelle2> **WHERE** <Tabelle1>.<Spalte> = <Tabelle2>.<Spalte>, [...])

Subquery als **SELECT** in **FROM** als Alternative zu Datenbanktabelle/View:

SELECT <Spalte_Alias> **FROM** (**SELECT** <Spalte> **AS** <Spalte_Alias> **FROM** <Tabelle>) **AS** <SELECT_Alias>

Subquery als **SELECT** in Feldliste:

SELECT <Spalte_Alias> ,

(**SELECT** <Spalte> **FROM** <Tabelle2> **WHERE** <Tabelle1>.<Spalte> = <Tabelle2>.<Spalte>, [...]),

[...]

FROM <Tabelle1>

SQL – Subquery – Beispieldaten

Kurs	Name
WI SE	Klaus
WI SE	Julia
INF	Benjamin

Studierende und
deren Kurs

Note der Studierenden
pro Vorlesung

Name	Vorlesung	Note
Klaus	Datenbanken	2,5
Julia	Datenbanken	2,4
Benjamin	Datenbanken	2,5
Klaus	Mathematik	1,8
Julia	Mathematik	1,6

SQL – Subquery– Anwendungsbeispiel

Fall 1

Alle Studierenden, deren höchste Note kleiner als 2,5 ist werden zu einem Bewerbungsgespräch eingeladen.

```
SELECT s.Kurs, s.Name
FROM Student AS s
WHERE Name IN
( SELECT Name FROM Note AS n WHERE s.Name = n.Name GROUP BY Name HAVING MAX(Note) < 2.5 )
```

Fall 2

Wie ist die durchschnittliche Note aller Vorlesungen und Studierenden basierend auf den durchschnittlichen Noten pro Vorlesung.

```
SELECT AVG(Avg_Note) AS Avg_Note
FROM ( SELECT AVG(Note) AS Avg_Note FROM Note GROUP BY Vorlesung ) AS Avg_Note
```

SQL – Subquery Operatoren

SELECT <Spalte> **FROM** <Tabelle1>

WHERE [**NOT**] **EXISTS**

(**SELECT** <Spalte> **FROM** <Tabelle2> **WHERE** <Tabelle1>.<Spalte> = <Tabelle2>.<Spalte>, [...])

EXISTS → Überprüfung, ob überhaupt ein Eintrag existiert

SELECT <Spalte> **FROM** <Tabelle1>

WHERE <Spalte> = | < | > | != | <= | >= **ANY** | **ALL**

(**SELECT** <Spalte> **FROM** <Tabelle2> [**WHERE** <Tabelle1>.<Spalte> = <Tabelle2>.<Spalte>, [...]])

ANY → Überprüfung, ob irgendein Eintrag abhängig des Operators zutrifft

ALL → Überprüfung, dass alle Einträge abhängig des Operators zutreffen

SQL – Subquery Operatoren – Beispieldaten

Kurs	Name
WI SE	Klaus
WI SE	Julia
INF	Benjamin

Studierende und
deren Kurs

Note der Studierenden
pro Vorlesung

Name	Vorlesung	Note
Klaus	Datenbanken	2,5
Julia	Datenbanken	2,4
Benjamin	Datenbanken	2,5
Klaus	Mathematik	1,8
Julia	Mathematik	1,6

SQL – Subquery Operatoren – Anwendungsbeispiel

✓ Fall 1

Alle Studierenden, die mindestens eine Note besser als 2 haben, werden gesondert geehrt.

Variante 1

```
SELECT s.Kurs, s.Name
FROM Student AS s
WHERE Name = ANY ( SELECT Name FROM Note AS n WHERE Note <= 2 )
```

Ergebnis

Kurs	Name
WI SE	Klaus
WI SE	Julia

Variante 2

```
SELECT s.Kurs, s.Name
FROM Student AS s
WHERE EXISTS ( SELECT Name FROM Note AS n WHERE s.Name = n.Name AND Note <= 2 )
```

SQL – Subquery Operatoren – Anwendungsbeispiel

Fall 2

Alle Studierenden, die nur Noten besser als 2,5 haben, sollen speziell genannt werden.

Korreakter Ansatz

```
SELECT s.Kurs, s.Name
FROM Student AS s
WHERE 2.5 > ALL ( SELECT Note FROM Note AS n WHERE n.Name = s.Name )
```

Ergebnis

Kurs	Name
WI SE	Julia

Falscher Ansatz

```
SELECT s.Kurs, s.Name
FROM Student AS s
WHERE Name = ALL ( SELECT Name FROM Note AS n WHERE Note <= 2.5 )
```

Dieser Ansatz führt dazu, dass der Name allen Namen entsprechen muss (also z.B. Julia = Benjamin & Julia = Klaus), dies kann nicht funktionieren.

Aufgabe 12

Aufgabe 12

Primärschlüsselprüfungen oder Fremdschlüsselprüfungen und deren Verletzungen dürfen für diese Aufgaben nicht umgangen werden. Auch wenn andere Möglichkeiten bestehen, sind alle Aufgaben mit Subselects zu lösen.

1. Ermitteln Sie, das höchste der Durchschnittsgehälter pro Ort.
2. Welche Attraktionen werden von Lisa Weber betreut?
3. Welche Besucher haben mindestens eine Attraktion besucht?
4. Welche Mitarbeiter arbeiten in Berlin?
5. Welche Eintrittskategorien werden nicht genutzt?

SQL – WITH

SQL –WITH

Alternative zu SUBSELECT, führt Queries in definierter Reihenfolge aus und ist weniger komplex zu lesen. Die Idee ist wie bei SUBSELECT mit Zwischenergebnissen zu arbeiten und so z.B. den Durchschnitt von Summen berechnen zu können.

WITH <Name der Laufzeittabelle> **AS** (<SQL Query>)

SELECT <Spalte> | * **FROM** <Name der Laufzeittabelle>;

WITH erzeugt zur Laufzeit eine Tabelle, die das Ergebnis der SQL Query enthält, davon kann direkt danach dann selektiert werden. Wichtig ist, dass WITH alleine kein eigenständiger Befehl ist und daher das Semikolon für das Ende eines Befehls erst nach dem SELECT gesetzt werden darf.

SQL – WITH – Anwendungsbeispiel

Die Bewertungen von Klausuren werden pro Matrikelnummer in einer Datenbanktabelle abgelegt. Ermittelt werden soll der Durchschnitt über alle Matrikelnummer über die Summe der jeweiligen Punkte, also wie viele Punkte wurden im Durchschnitt für eine Matrikelnummer über die gesamte Studienzeit erreicht.

Schritt 1 – Erstellen der WITH-Tabelle mit Summe der Punkte pro Matrikelnummer

```
WITH Noten_Studierende AS (  
    SELECT Matrikelnummer, SUM( Punkte ) AS Klausur_Punkte  
    FROM Klausurbewertung  
    GROUP BY Matrikelnummer  
)
```

Schritt 2 – Berechnen der Durchschnittspunkte aus der WITH-Tabelle

```
SELECT AVG( Punkte ) AS Durchschnitt FROM Noten_Studierende ;
```

Aufgabe 13

Aufgabe 13

Lösen Sie Aufgabe 12 ohne Subselect und stattdessen mit WITH.

SQL – Stored Function

SQL – Function

Eine Stored Function ermöglicht es eigene Funktionen zu definieren. Diese können dann in z.B. SELECT Queries verwendet werden. Functions besitzen nur eine einzige Rückgabe und ermöglichen nur begrenzt die Nutzung von SQL Queries innerhalb der Function.

DELIMITER \$\$

CREATE FUNCTION [**IF NOT EXISTS**] <Funktionsname>

([<Parameter> <Datentyp> [...]])

RETURNS <Datentyp> [**[NOT] DETERMINISTIC ...**]

BEGIN

<SQL>

RETURN <Ergebnis>

END

\$\$

DELIMITER → setzt Zeichen für Befehlsende

DELIMITER ;

DECLARE → Deklaration einer Variablen

SET → Setzen einer Variablen

<SQL> (Beispiel):

DECLARE Alter INT;

SET Alter = TIMESTAMPDIFF(YEAR, Geburtsdatum, Referenzdatum);

Anzeige der existierenden Funktionen:

SHOW CREATE FUNCTION

<Datenbank>.< Funktionsname>;

SHOW FUNCTION STATUS

WHERE Db = <Datenbank>;

Löschen einer Funktion:

DROP FUNCTION [**IF EXISTS**] <Funktionsname>;

IF EXISTS | IF NOT EXISTS können auch bei Functions genutzt werden.

SQL – Function – Beispiel-Code

Verzweigung:

```
IF Referenzdatum > Heute THEN  
    SET Referenzdatum = Heute;  
END IF;
```

Schleife:

```
DECLARE Summe INT DEFAULT 0;  
DECLARE i INT DEFAULT 1;  
  
WHILE i <= n DO  
    SET Summe = Summe + i;  
    SET i = i + 1;  
END WHILE;
```

DECLARE

SET

WHILE

CASE

IF

→ Deklaration einer Variablen

→ Setzen einer Variablen

→ Schleife mit Bedingung

→ Fallabfrage

→ Verzweigung

Fallabfrage:

```
DECLARE Kategorie VARCHAR(20);
```

```
SET Kategorie = CASE  
    WHEN alter < 13 THEN 'Kind'  
    WHEN alter < 18 THEN 'Teenager'  
    WHEN alter < 65 THEN 'Erwachsen'  
    ELSE 'Rentner'  
END;
```

SQL – Function – Anwendungsbeispiel

Zu einem Referenzdatum soll mit Hilfe des Geburtsdatums das Alter ermittelt werden.

```
DROP FUNCTION IF EXISTS calculate_age;
DELIMITER //
CREATE FUNCTION IF NOT EXISTS
    calculate_age(Birthdate DATE, Reference_Date DATE)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE today DATE;
    SET today = CURDATE();
    IF Reference_Date > today THEN
        SET Reference_Date = today;
    END IF;
    RETURN TIMESTAMPDIFF(YEAR, Birthdate, Reference_Date);
END //
DELIMITER;

SELECT calculate_age('2000-01-01', '2030-01-01');
```

Aufgabe 14

Aufgabe 14 ✓

Erstellen Sie eine Funktion, welche die Eintrittskategorie abhängig des Geburtsdatums ermittelt.

SQL – Stored Procedure

SQL – Procedure

Eine Stored Procedure ermöglicht es eigene Prozeduren zu definieren. Diese können dann z.B. verwendet werden um vordefinierte Abläufe zu vereinheitlichen. Procedures besitzen eine beliebige Anzahl Rückgaben und ermöglichen die Nutzung von SQL Queries innerhalb der Procedure. Procedures können nur separat mit CALL gerufen werden.

DELIMITER \$\$

```
CREATE PROCEDURE [ IF NOT EXISTS ] <Prozedurname>  
([ [ IN | OUT | INOUT ] <Parameter> <Datentyp> [...]])  
[ [ NOT ] DETERMINISTIC ...]  
BEGIN  
<SQL>  
END;  
$$
```

DELIMITER ;

@ → Escape Zeichen für Rückgabe-Parameter

Aufruf:

```
CALL generate_random_numbers(<Eingabe>, @<Rückgabe>);  
SELECT @<Rückgabe> AS <Spaltenname>;
```

Anzeige der existierenden Prozeduren:

SHOW PROCEDURE STATUS

WHERE Db = <Datenbank>;

SHOW CREATE PROCEDURE

<Datenbank>.<Prozedurname>;

Löschen einer Prozedur:

DROP PROCEDURE [**IF EXISTS**] < Prozedurname >;

IF EXISTS | IF NOT EXISTS können auch bei Functions genutzt werden.

SQL – Procedure – Anwendungsbeispiel

Abhängig einer Anzahl definierter Vorgänge sollen Zufallszahlen ermittelt werden. Diese sollen sowohl jeweils angezeigt als auch aufsummiert werden. Die Summe soll als Ergebnis zurückgegeben werden.

```
DROP PROCEDURE IF EXISTS generate_random_numbers;
DELIMITER //
CREATE PROCEDURE generate_random_numbers(
    IN Counter INT,
    OUT Sum INT
)
BEGIN
    DECLARE i INT DEFAULT 0;
    DECLARE Random_Number INT;
    SET Sum = 0;

    WHILE i < Counter DO
        SET Random_Number = FLOOR(1 + (RAND() * 100));
        SELECT Random_Number AS "Random Number";
        SET Sum = Sum + Random_Number ;
        SET i = i + 1;
    END WHILE;
END;
//

DELIMITER ;

CALL generate_random_numbers(5, @Sum);
SELECT @Sum AS " Sum of random numbers";
```

Aufgabe 15

Aufgabe 15

Erstellen Sie eine Prozedur, welche die Einstellung eines neuen Mitarbeiters darstellt, dabei soll Vorname, Nachname und Geburtsdatum in die Prozedur gegeben werden.

Die Adresse ist die Unternehmensadresse und daher immer gleich.

Der neue Mitarbeiter soll eine zufällige Anzahl Attraktionen betreuen und das Mindestgehalt plus eine zufällige Zahl zwischen 100 und 200 mehr erhalten.

Das neue Gehalt soll als Rückgabe definiert werden und danach angezeigt werden.

SQL – Trigger

SQL – Trigger

Trigger können genutzt werden um für bestimmte SQL Befehle automatisch weitere Befehle auszuführen, z.B. ein Datum setzen oder einen zusätzlichen Log-Eintrag schreiben.

```
CREATE TRIGGER [ IF NOT EXISTS ] <Trigger_Name>
AFTER | BEFORE
INSERT | UPDATE | DELETE
ON <Datenbanktabelle>
FOR EACH ROW
BEGIN
    <SQL>
END;
```

Anzeigen der existierenden Trigger:

```
SHOW TRIGGERS FROM <Datenbank>;
```

Löschen eines Trigger:

```
DROP TRIGGER [ IF EXISTS ] <Trigger_Name>;
```

IF EXISTS | **IF NOT EXISTS** können auch bei Functions genutzt werden.

SQL – Trigger – Beispieltabellen

Studierende

ID	Name	Matrikelnummer
...	...	...
...	...	...
...	...	...

Mensakarte

ID	Student_ID	Guthaben	Erstelldatum
...	...	...	
...	...	...	
...	...	...	

Bibliotheksausweis

ID	Student_ID	Ausweisnummer	Erstelldatum
...	...	...	
...	...	...	
...	...	...	

SQL – Trigger – Anwendungsbeispiel

Wenn jemand der Universität beitrtritt, soll neben de Matrikelnummer zusätzlich auch direkt und automatisch ein Ausweis für die Bibliothek und eine Karte für die Mensa angelegt werden.

DELIMITER \$\$

CREATE TRIGGER Neue_Person_an_Universitaet

AFTER INSERT ON Studierende

FOR EACH ROW

BEGIN

INSERT INTO Bibliotheksausweis (Student_Id, Ausweisnummer, Erstelldatum)
VALUES (NEW.id, CONCAT('BIB-', NEW.id), CURRENT_DATE());

INSERT INTO Mensakarte (Student_Id, Guthaben, Erstelldatum)
VALUES (NEW.id, 10.00, NOW());

END \$\$

DELIMITER ;

INSERT INTO Studierende (Name, Matrikelnummer)
VALUES ('Katharina Baumann', '123456789');

NEW ist ein spezielles Schlüsselwort in MySQL-Triggern, das sich auf die neuen Werte einer Zeile bezieht – OLD hingegen bezieht sich auf die alten Werte.

Befehl	NEW	OLD
INSERT	X	
UPDATE	X	X
DELETE		X

Automatisch erzeugt werden:

→ Ein Bibliotheksausweis mit BIB-<Id>

→ Eine Mensakarte mit Guthaben 10

Aufgabe 16

Aufgabe 16

Erzeugen Sie einen Trigger, der automatisch die passende Kategorie berechnet, sobald ein neuer Besucher hinzugefügt wird.

Fragen?

Mail: andreas.nicola@sap.com